

A perfect flow field on a raster is worth no initialisation: separating representation, region and accuracy in RANS warm starting

Ali Jabbary^{a,*}, Kasra Ghanavati^b

^a*Department of Mechanical Engineering, Urmia University, Urmia, Iran*

^b*School of Computing and Mathematical Sciences, University of Greenwich, London, United Kingdom*

Abstract

Neural surrogates for external aerodynamics are usually evaluated as predictors. Used instead as initial conditions for a RANS solver, what decides whether they help separates into three properties: where the surrogate stores its field, which region it covers, and how accurate it is. We separate them without a network, by handing a wall-resolved simpleFoam solve its own converged field back in controlled variants, so no result rests on a model's accuracy claim. Every figure is viscous-drag convergence over thirteen cases, paired within case.

Read at the solver's own cell centres, the exact field saves 93.6% of a cold solve (95% CI 92.9 to 94.3, 13 of 13). Stored first on a 128^2 Cartesian raster – the format grid-based neural operators emit – it saves 3.4% (CI -2.3 to +8.8, $p = 0.27$): indistinguishable from no initialisation. **A body-fitted grid holding the same 16,384 values costs nothing measurable.** Restricting the field to the boundary layer costs 20.7 points and using a surrogate's prediction a further 54.5, so **accuracy matters, and more than region does** – contradicting an earlier form of this claim, which we withdraw.

Nor is it the size of the error: a smooth perturbation carrying the raster's own error norm costs 12.2 points at that band where the raster costs 82.5. What costs is **where** a format's error sits, and a parameter-free closed form separates the two formats by y^+ alone before any solve. That quantity is **not** the mediator: three ways of restoring the near-wall state leave convergence unchanged, and a seed worse on every local measure we can make converges no worse. Classical grid sequencing beats the learned seed on both axes.

Keywords: warm start, Reynolds-averaged Navier–Stokes, neural operator, convergence, acceleration, boundary layer, OpenFOAM

Highlights

- The same converged field: +93.6% read at the solver's points, +3.4% off a raster
- A perfect flow field on a 128^2 raster is worth no initialisation ($p = 0.27$)
- Body-fitted grids are not rasters: same 16,384 values, and no measurable cost
- Representation, accuracy and region priced at -90.2, -54.5 and -20.7 points

*Corresponding author.

Email address: `st_a.jabbary@urmia.ac.ir` (Ali Jabbary)

- Three ways of restoring the near-wall state all fail to recover the solve

1. Introduction

A neural surrogate that predicts a flow field in 30 ms is not, by itself, a faster CFD pipeline. Engineering decisions are made on force coefficients with error bars, and a surrogate whose drag is 5% off does not replace a solve — it precedes one. The natural role for it is therefore as an **initial condition**: if the surrogate’s field is closer to the answer than a uniform freestream, the solver should need fewer iterations to get there, and the saving is free of any accuracy claim, because the converged answer is the solver’s, not the network’s.

That argument is clean and, as stated, wrong. This paper is about why, about the one number that decides it, and about what to do when that number comes out badly.

Our starting configuration is deliberately unfavourable to shortcuts. The solver is unmodified OpenFOAM v2606 [11] `simpleFoam` with the Spalart–Allmaras model [10]. The mesh is a wall-resolved body-fitted C-grid, 31,700 cells, first cell centre $5 \cdot 10^{-6}$ chords off the wall, aspect ratios up to $2 \cdot 10^5$. The Reynolds number is $3 \cdot 10^6$ — flight, not a demonstration. The surrogate is a Transolver-style point model trained on AirfRANS, evaluated on airfoils and angles it did not see. The metric is not a residual threshold: it is the number of iterations before a force coefficient enters and stays inside a band around its converged value, because that is the quantity an engineer actually waits for.

The failure, with accuracy removed from the experiment

Take the **exact converged flow field** — not a prediction, the answer itself — store it the way neural-operator outputs are normally stored, and hand it back to the solver as an initial condition. On a 128^2 Cartesian raster, total-drag convergence is **548% slower** than starting from uniform freestream. On an equal-budget wall-fitted 256×64 grid, **182% slower** (both $C_d@1\%$, five cases). A perfect answer, stored in the field’s standard output format, is a *worse* initial condition than no initialisation at all.

Those two figures are the study’s most dramatic and they are **not** the ones this paper’s claims rest on. Total drag fails the readability rule of §4 on the thirteen-case corpus, and §5.1 reports the headline on viscous drag instead, where the same contrast is +93.6% against +3.4% and carries a p-value. We lead with total drag here because it is where the effect is largest and most legible, and we say immediately that it is not where the effect is *established*.

Because the field being stored is exact, nothing about this depends on how accurate any network is, how it was trained, or which architecture it uses. The defect is in the **representation**.

It is not resolution, and the arithmetic says so

The obvious response is to spend more values. We measured that too — a ladder of uniform rasters from 128^2 to 421^2 of the same exact field — and the saving is flat and negative throughout (§7.1).

The reason is that the failure is not one of resolution but of **placement**, and it is arithmetic rather than statistical. When a field is resampled through a grid, every mesh cell nearer the wall than the grid’s first wall-normal station receives the value belonging to that station; the representation holds no sample in between. Viscous drag is the integral of the tangential velocity gradient in the first cell — 60–84% of total drag in these cases — so the reconstructed gradient is not degraded but *replaced*,

and overestimated by

$$G = u^+(y_1^+) / u^+(y_-c^+)$$

where y_1^+ and y_-c^+ are the wall-unit positions of the representation's first station and the mesh's first cell centre. This is the law of the wall and nothing else; **it contains no fitted parameter**. Measured over five cases at five first-station heights spanning a factor of fifty, it **bounds the damage above** in every row where its mechanism is active, by between $1.9\times$ and $2.8\times$ (§6.2).

Two things follow immediately. u^+ grows *logarithmically*, so a 10.8-fold increase in stored values moves the predicted damage from $35.7\times$ to $35.5\times$ — which is why the resolution ladder is flat, and why it is flat by a computable amount. And the fix is a grading choice rather than a budget: a 64-level geometric stack whose first station lies inside the first cell needs a growth ratio of 1.214, an ordinary mesh. **A 512^2 raster holds 262,144 values and still fails; a wall-fitted grid of 8,192 values passes** (§6.5).

Contributions

Ordered by what we think survives, not by what is largest. §10 collects in one place the five claims this project made and then withdrew.

1. **A representational failure of the field's standard output format, measured with the accuracy confound removed** (§1, §7.1). The exact converged answer, stored as a raster, is a catastrophically bad initial condition, and refining the raster cannot fix it. A body-fitted grid of the *same* budget is not, which is what makes this a statement about placement rather than about grids.
2. **A decomposition into properties that can be set independently** (§5.1), each priced by a control that moves one variable on $C_{d,v}@1\%$: region costs 20.7 points, a raster representation 90.2, a body-fitted one nothing measurable, and accuracy 54.5 — all on the thirteen-case corpus, from one tree, 13 of 13 cases where the effect is real. It replaces the claim that representation matters *rather than* accuracy, which our own control contradicts and which we withdraw.
3. **A closed-form criterion for it, with no fitted parameter**, computable from a mesh and an output format before any solve is run, and a measured *upper bound* on the damage across a fifty-fold range of first stations (§6.2). We ship it as a command-line tool so it can be run on a mesh and a format that have nothing to do with this study (§6.4).
4. **The criterion holds across Reynolds number**, checked from $Re\ 10^3$ to $3\cdot 10^6$ at no compute cost, including its counterintuitive prediction that the same representation on the same mesh does *more* damage at lower Reynolds — and a measured statement of the regime where it stops being quantitative (§6.6).
5. **Three independent demonstrations that the near-wall state is not the mediator** (§5.2.1, §5.5, §6.7). Moving the representation's first station inside the mesh's first cell takes the gradient error to **1.8%** and the roughness to the converged field's own — and convergence gets *worse*, winning 0 of 5. Repairing the gradient by wall function moves convergence 0.1 points; smoothing that repair moves it 0.6 more. We predicted the opposite, in writing, before running any of it. What the damage *is* instead we have narrowed to the seed's smooth, outer, globally elliptic content and not isolated (§5.9).
6. **A recipe with three necessary conditions**, each isolated by a controlled arm changing one variable on one prediction, none sufficient alone (§5.2–§5.4), and **generality at $n = 13$** — +18.4%

on viscous drag, 13/13 cases, $p = 0.0002$, with a passing oracle control and a null negative control (§5.1).

7. **A comparison with the classical warm start** the machine-learning initialisation literature does not make: grid sequencing, charged honestly (§5.7). It is better than the learned seed **on both quality and price, in fine-mesh-equivalent iterations**: +75.9% on viscous drag against +14.6%, and $\approx 68\%$ end-to-end once its coarse solve is charged at that solve’s own convergence (~ 53 fine-equivalent) rather than at a full budget. Both figures are iteration counts; we do not have matched wall-clock for grid sequencing and do not claim it. The criterion predicts this correctly for a method containing no network. We report it because a paper that measures a learned seed only against a cold start has not answered the question a practitioner asks, and because an earlier draft of this work reached the opposite conclusion from a charge it had set too high.
8. **The control for the objection this design invites** (§5.9). An oracle seed is a fixed point of the discrete operator, so any contrast against it might be measuring distance from that fixed point rather than representation. A smooth perturbation carrying the raster’s *own* whole-field error norm, ramped to zero inside the boundary layer, costs **12.2 points** where the raster costs **82.4** — so roughly six sevenths of the raster’s harm is attributable to where its error sits rather than how large it is.
9. **An acceptance certificate** bounding the worst case at $(1 + K/N) \times$ cold with a rule that never sees a cold run (§8), and **three falsified predictions** a reader would otherwise make (§7).

What this paper does not claim

It does not claim a speed record, and it is not competing for one. The largest number in this literature is $26.3\times$ [8]; ours is +18.4%. §2.1 and §7.3 explain by measurement rather than rebuttal why that number and ours concern different regimes — an oracle seed of the entire wake buys +0.5% here, and their own result is reported as conditional on an accurate near-body field supplied separately. A reader who wants the fastest warm start should read that paper; a reader who wants to know whether *their* surrogate can warm-start *their* solver, and to find out before paying for it, should read this one.

It does not claim the surrogate is accurate — accuracy is a separate question and irrelevant here, since the solver converges to its own answer regardless. And it does not claim generality of the *demonstration* beyond 2-D incompressible steady RANS with one turbulence model on NACA 4-digit sections. The **criterion** is more general than that by construction, since it is a statement about wall units and sampling positions rather than about any solver or model, but we measured its consequences in one place and §10 says so plainly.

2. Related work

2.1 Warm-starting a flow solver with a learned field

Using a network’s prediction as a solver’s initial condition is an active line with a wide spread of reported gains, from about $2\times$ to $26\times$. The spread is usually read as a difference in method quality. **We read it as a difference in representation and in which region is seeded, and §6 gives the criterion that sorts it.**

Zhou et al. [12] map a low-fidelity potential-flow solution to a RANS field with an equivariant vector-cloud operator and use the result to start `rhoSimpleFoam` with Spalart–Allmaras on wall-resolved unstructured meshes at $Re = 6\cdot 10^6$ — a configuration close to ours. They report about $2\times$ on the

residual and, on the metric this paper also uses, **11× to reach 1% force error and 16× to reach 5%**. Their operator is a *region-to-point* map evaluated at a target point, so it is mesh-native by construction, and the criterion in §6 says a mesh-native seed retains the wall gradient. Their result is the largest near-wall warm-start gain in the literature and it is consistent with, not contrary to, what we measure.

Fuchi et al. [8], whose group also studies multi-fidelity learned flow models [22], report the largest number in this literature — **26.3× fewer iterations and 16.4× wall-clock** — from a convolutional wake-extension model. It is worth being precise about what that result contains, because at first reading it dwarfs everything here. Their method divides the domain into near-body, wake and off-body regions; the network predicts the *wake*, and the acceleration is reported to be achieved "when combined with an accurate flow prediction in the near-body region". The near-wall state is therefore supplied already correct, and the network's contribution is the region where no near-wall gradient exists to lose. §7.3 measures the complementary bound in our configuration: an **oracle** seed of the exact converged field across the entire downstream region is worth **+0.5%** on viscous drag here. The two results are about different regimes and compose rather than compete.

Sharpe et al. [7] initialise transient URANS with a point-based model combined with potential flow and report $\sim 2\times$, using a drag-band metric close to ours; we measure **potentialFoam** alone as an arm and find it inert on drag (+0.6%). Hu et al. [13] predict at cell and wall-face centroids — mesh-native again — for a submarine hull and report $3.5\times$ at a residual threshold of $5\cdot 10^{-6}$, with cross-mesh generalisation. **Their validation is residual-based throughout and reports no force coefficient.** That is the common practice in this literature, and §5.6 is the reason we departed from it: on our configuration one seed is **+22.1% on the residual and -58.8% on total drag**. We are not claiming their result is wrong; we are reporting that on our cases the two metrics can disagree in sign, so we fixed a force metric before running the arms.

A third line puts the surrogate *inside* the solver rather than ahead of it: Sousa et al. [19] embed one in the PISO pressure-velocity loop and, importantly for §9, introduce a solver-intrinsic, hardware-independent measure of effort that charges the surrogate's own overhead — the same accounting concern that makes us report iterations and seconds separately.

At the level of the linear algebra rather than the field, Nows [4] supplies learned initial guesses to Krylov solvers and reports up to 90% time reduction; Oh et al. [5] constrain learned corrections so Newton convergence is preserved, reporting $5.4\times$ at 6.4M DOF; Khodak et al. [6] select preconditioners online. These are complementary to and composable with an outer-field seed. Oh et al. are the closest prior art to our §6 in *spirit* — both say that L^2 accuracy is not what makes a seed good — and it is worth stating the difference plainly: theirs is a spectral property of the Jacobian, established for Newton solvers; ours is a named, measured, geometric defect of the *representation* (the first-cell wall gradient), with a closed form and a pre-flight test that needs no solver internals.

2.2 The near-wall region is independently known to be where these models fail

The mechanism we measure is not a surprise to the prediction community; what is new is its consequence for warm starting.

The AirfRANS benchmark paper [1] reports that models "have difficulties predicting wall shear stresses as velocity values at the closest nodes from the geometry are often largely overestimated", and identifies this as what damages the drag coefficient. §6 measures the same overestimate — a factor of ~ 20 — arising from the *representation alone*, with the exact converged field in place of any prediction.

The field audits itself in the same direction: benchmark evaluations of current architectures for aerodynamic prediction [20] and the ML4CFD competition retrospective [21] both report near-wall quantities as the weak point.

DD-RNO [14] argues that "a single neural architecture cannot simultaneously resolve sharp near-wall boundary layers and smooth far-field potential flow" and routes query points to separate inviscid, boundary-layer and wake decoders by wall distance. Zhang et al. [15] replace isotropic proximity modelling with explicit tangential–normal structure for the same reason. Both are motivated by prediction accuracy. That an independent line arrives at a *region split by wall distance* is convergent evidence for condition 2 of §5.3, reached from the other direction.

2.3 Mesh-native surrogates

Transolver [2], PCNO [3] and their successors [16, 17, 23] predict on native mesh points rather than on a raster, and the capability is presented as an accuracy or memory convenience. This paper’s claim is that it is not a convenience: it is the difference between a seed that accelerates a solve and one that costs more than starting cold. A surrogate stored as a 128^2 image cannot be used for warm starting at all, and no amount of raster refinement recovers it (§7.1).

The point sharpens as the field moves to pretrained, general-purpose aerodynamic models [24], where the output representation is a design decision taken once and inherited by every downstream user. If such a model is ever to be handed to a solver, the criterion of §6 is a constraint on that decision.

2.4 Classical initialisation, which is what a practitioner actually uses

The comparator that matters is not a uniform freestream. Production aerodynamics warm starts by **grid sequencing**: solve on a coarsened mesh, map the result up, continue on the fine mesh — shipped in OpenFOAM as `mapFields`, and closely related to full-multigrid initialisation, which is standard in structured compressible codes and still an active line in its own right [26]. A learned initialisation measured only against a cold start has not answered the question a practitioner asks.

We therefore run grid sequencing as an arm (§5.7), with the coarse solve charged in fine-mesh-equivalent iterations. It is also the criterion’s out-of-sample test: a coarsened body-fitted mesh keeps its wall-normal stations clustered at the wall, so §6 **predicts** that grid sequencing preserves the first-cell gradient and helps — a prediction about a method containing no network, no training data and no surrogate, and one the criterion was not derived from.

2.5 Fallbacks and worst-case guarantees

Preserving worst-case behaviour by falling back to the classical method when a learned component is untrustworthy is an established pattern; Yavlovich et al. [9] apply it to linear assignment with a dual warm start and retain baseline runtime even at 100% fallback, and Schmidtobreck et al. [18] warm-start active-set solvers with a GNN while retaining convergence guarantees. **We cite this as prior art for the pattern.** What is ours is its instantiation for a PDE solver’s initial *field*: a decision rule that reads only a short probe of the solve it is about to commit to, leave-one-case-out calibration, and a measured capture-versus-cost curve showing that longer probes are monotonically worse (§8). We note that Zhou et al. [12] report exactly the failure such a rule exists to catch — in their extrapolation case the initialisation gives no clear advantage and the residual sharply increases — and have no test for it.

3. Setup

Solver. OpenFOAM v2606 (ESI), `simpleFoam`, steady incompressible SIMPLEC (`consistent yes`), Spalart–Allmaras. `nNonOrthogonalCorrectors 2`. Under-relaxation `U 0.7`, `nuTilda 0.4` — chosen, not defaulted; see §4 rule 2. Budget 6000 iterations. Every case is instrumented with both the `forceCoeffs` and `forces` function objects, so total, pressure and viscous drag are available separately at every iteration.

Mesh. Body-fitted C-grid generated by `blockMesh` from a specification we control (`solver/cgrid.py`): 31,700 cells, 20-chord far field, stitch-free wake cut via shared vertex ids, first cell $1e-5$ chords tall (nominal centre $5e-6$, measured minimum wall distance $3.79\text{--}4.01e-6$ depending on section), wall-normal grading to $y^+ < 1$. The same mesh is used for every arm of every experiment, so mesh quality is never a differential effect.

Verification (`scripts/verify_mesh.py`, Table 1). Every result in this paper is a *ratio* — iterations from one seed against iterations from a cold start on the same mesh with the same discretisation — so mesh-independence of the absolute force is not what the claims rest on. It is still what should be established before they are read, and one part of it bears directly on which metric the paper is entitled to use.

The mesh is wall-resolved, measured rather than asserted. Over every surface station of every case, y^+ at the first cell centre has median $0.59\text{--}0.63$ and maximum 1.12 ; between 0% and 7.5% of stations exceed 1 . Spalart–Allmaras is run with no wall function, so this is the condition that has to hold.

Grid convergence, against a systematically coarsened member of the same family (every count halved, first cell doubled: 7,850 cells). Changing the cell count by a factor of four moves the forces by:

quantity	mean absolute change, coarse against study mesh
<code>C_d,v</code> viscous drag	6.9%
<code>C_l</code> lift	4.0%
<code>C_d</code> total drag	20.2%
<code>C_d,p</code> pressure drag	49.8%

The quantity this paper reports its headline on is the one least sensitive to the mesh, and the quantity it declares secondary is the one most sensitive. That ordering is not something we arranged: `C_d,v` is a near-wall integral that this grading resolves, and `C_d,p` is a global elliptic quantity that a 7,850-cell mesh does not. It is independent support for §4’s readability rule, which reaches the same conclusion from convergence behaviour alone.

The absolute number. `C_d` = 0.00916 for NACA0012 at 0° , $\text{Re } 3\cdot 10^6$. Fully turbulent Spalart–Allmaras with no transition model should sit at or slightly above section-data values of ≈ 0.0085 , and it does, by about 7% . We do not claim this mesh is grid-converged in the absolute — the coarse-level comparison above shows the family is still moving — and no claim in this paper requires it to be.

Cases. NACA 4-digit sections at $\text{Re} = 3e6$, `u_inf = 1`, kinematic pressure. Two sets. The **mechanism study** is 5 cases (0012@ 4° , 2412@ 2° , 0015@ 6° , 0012@ 0° , 2415@ 5°) carrying fifteen seeding arms, which is where every controlled contrast and the acceptance gate are measured. The **generality corpus** is 13 cases (0012 at $8/10/12^\circ$, 2412 at $8/10^\circ$, 0018 at $4/8^\circ$, 4415 at $2/4^\circ$, 2415 at $2/8^\circ$, 0015 at $2/4^\circ$) carrying four arms, which is where the headline is measured. The two sets are disjoint, and every number states which it came from.

Surrogate. A Transolver-style point model, $(B,N,7) \rightarrow (B,N,4)$, trained on AirfRANS under the conventions of the companion paper (dimensional fields, $\nu = 1.56e-5$, Re from $|u_{in}|$). It is queried directly at the C-grid cell centres — `solver/surrogate_seed.py` — with no intermediate representation of any kind. Its training `sdf` distribution is centred on 0.23 chords, which is why seeds are cut off at 3.5 chords and why the outer field is left cold.

Trees, and why the same arm reads two numbers. The study is seven separate solve trees, each with its own case set and its own arm set:

tree	cases	arms	used in
corpus	13	5	§5.1, §5.2, §5.1 — the headline and the decomposition
repr3	6	15	§5.3, §5.4, §5.6, §8
placement2	5	9	§5.2.1 (the placement ladder)
repair	5	8	§5.5
perturb	5	4	§5.9 (the fixed-point control)
sequencing	5	7	§5.7
wallclock2	5	4	§9
Reynolds ladder	2×5 Re	—	§6.6 (no solves; re-uses cold runs)

A saving depends on the arm set, because §4’s settled reference is a median over the arms that settled — so **the same arm reads slightly differently in different trees**, and `nf_bl` on `C_d@1%` is +33.9% in `repr3` and +34.1% in the placement tree. That is not noise being reported twice; it is the reference moving, and §4’s rule is that a number must name the arm set it was computed over. Every table below states its tree. Cross-tree comparisons of the *same* arm are not made anywhere in this paper; comparisons are always within a tree.

Arms. Every experiment carries a **converged-field oracle** as a control. The oracle is the cold run’s own converged solution, re-injected as an initial condition, and it must post a large positive saving before any other arm in that experiment is read. Across twelve experiments the control has read +68% to +99.9%. An experiment whose control fails is a broken measurement, and we say so rather than reporting its other arms.

4. How to measure a warm start without fooling yourself

This section is a contribution, not preamble. Each of the first six rules below changed a **sign** on this project’s own data; rules 7 and 8 were added by the thirteen-case sweep and each cost us a result. All are implemented in `solver/scoring.py` or `scripts/reanalyse_depth.py`, each with a test that names the mistake it prevents.

1. **A residual threshold measures a convergence rate only while the residual is still falling.** Report every threshold as a multiple of the run’s residual floor and refuse to read anything below $\sim 5x$. The same arm read +15%, +31% and +13% at 1.9x, 1.3x and 0.9x the floor — noise presented as a trend.
2. **The floor itself is usually an artifact, so find out which one.** Ours was under-relaxation at 0.9 with SIMPLEC, not the 2e5-aspect-ratio cells: `U 0.7 / nuTilda 0.4` moved the floor 30x, from $1.1e-5$ to $3.5e-7$. Ten other variants — longer budgets, tighter inner tolerances, a better wake mesh — bought nothing.
3. **An arm that never reaches the target must be bounded at its full budget, not**

dropped. Dropping non-finishers turned a true -199.4% into -31.2%: the failing arm was rewarded for failing.

4. **Score every arm against one external reference**, never against its own final value. Grading the oracle against itself turned a +73.5% control into +1.0%.
5. **Only arms whose coefficient has stopped moving may define that reference.** Peak-to-peak over the last tenth of the run, against a quarter of the band. A single diverged arm previously set the reference and condemned an entire table; restricting the reference to settled arms moved the spread from 3.104% to 0.334%.
6. **nNonOrthogonalCorrectors multiplies the pressure history.** Parse per Time block; zipping fields by index across a log made pressure lag velocity 3:1 and moved every shallow number.

One consequence of rule 5 is worth stating even though it is small here. Because the reference is a median over the arms that have settled, *adding* an arm to a tree re-scores every arm already in it. **A number must therefore name the arm set it was computed over**, and ours does (`-drop-arm` declares it). We measured the sensitivity rather than assuming it away: removing our most recently added arm moves every headline number by at most 0.5 percentage points.

The primary metric. `iterations_to_force_band`: the first iteration after which a coefficient stays within $\pm b$ of the reference for the rest of the run. Saving is 1 - warm/cold, bounded at the budget. We report $b = 1\%$, 0.5% , 0.2% and state which bands are readable.

It is a last-exit statistic and that is deliberate, but a reader is entitled to the alternative.

First entry — the first iteration at which the coefficient touches the band at all — is the natural competitor, and on this data it is useless in a way worth showing rather than asserting. A cold start sweeps from freestream to converged and passes *through* the band on the way, so its first entry is artifactually early: scored that way, `nf_b1` reads **-1645%** against cold, and every warm arm looks catastrophic. The two statistics differ by a median of 186 iterations and by as much as 1673 (78 case-arm pairs, `scripts/causal_stopping.py`), which is the size of the wandering that last-exit exists to charge and first entry ignores. Both are recorded per arm in `results/causal_stopping.json` so the choice can be audited.

It also needs the converged answer, which production does not have. That is acceptable for measuring a mechanism and not for measuring a workflow, so §5.1 re-scores every arm under a *causal* stopping rule — a flat trailing window, no reference and no cold run — and reports what changes. What changes is the recommendation, not the decomposition.

Readability. A band is readable only if the settled arms agree about the converged value to well inside it (rule 5) — concretely, to within half the band. On the core study the largest disagreement is 0.232% of Cd, so $b = 1\%$ and $b = 0.5\%$ are readable (limits 0.5% and 0.25%) and $b = 0.2\%$ is not (limit 0.1%). **The 0.5% verdict is thin:** 0.232% against a 0.25% limit. We report the margin rather than only the verdict, because one additional case can flip it.

Provisional. Every readability verdict in this paper is a property of the case set, since the reference is a median over cases and arms. The 13-case sweep recomputes all of them, and they are re-checked rather than inherited.

The saving depends on the band, and the two drag components differ. On the five-case mechanism study:

arm	Cd@1%	Cd@0.5%	Cd@0.2%	Cd_v@1%	Cd_v@0.5%	Cd_v@0.2%
nf_bl	+33.9%	-4.2%	-38.5% (<i>unreadable</i>)	+14.6%	+13.7%	+11.0% (<i>unreadable</i>)
oracle_mesh	+92.1%	+92.4%	+93.1%	+92.5%	+91.9%	+91.3%

Viscous drag is monotone across bands and total drag is not. Monotone stability *is* the evidence that a number is a convergence-rate measurement rather than an artifact of where a wandering curve happens to cross a line — and Cd_v is 60–84% of the drag here.

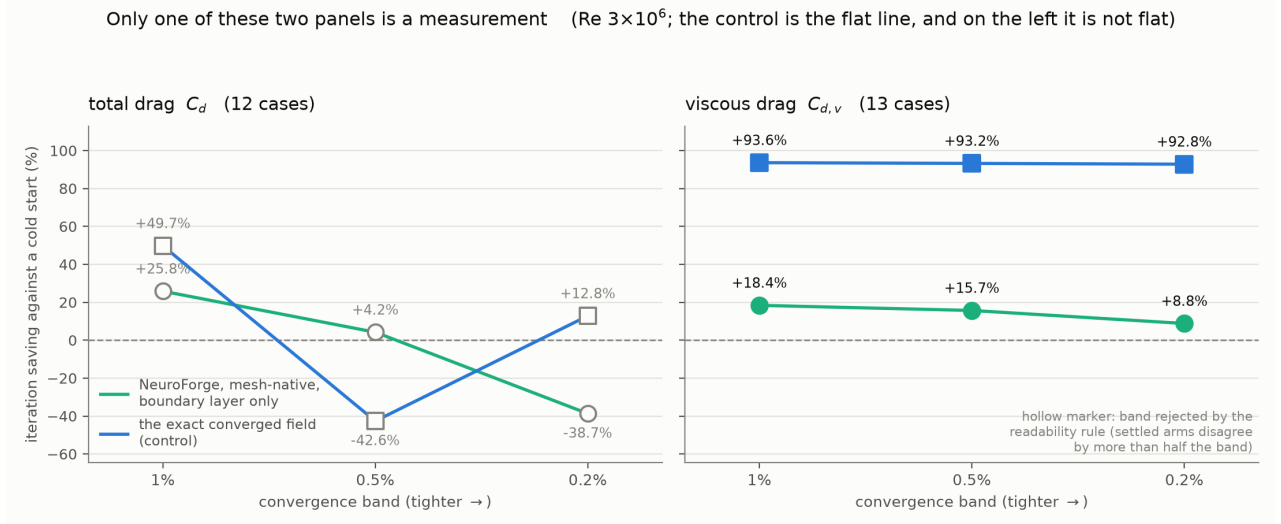


Figure 2. Only one of these two quantities is a convergence-rate measurement. Iteration saving against a cold start as the convergence band is tightened, on the thirteen-case corpus. On **total drag** (left) every band is rejected by the readability rule, and the converged-field control itself swings $+49.7\% \rightarrow -42.6\% \rightarrow +12.8\%$ — a control that is not flat indicates a measurement that cannot be read. On **viscous drag** (right) the control is flat at $+93\%$ and the trained seed is monotone at $+18.4\% / +15.7\% / +8.8\%$. Hollow markers are bands the readability rule rejected; they are plotted rather than deleted, because a curve with its last point removed would look steadier than the measurement is.

A withdrawal. A previously recorded $+41.8\%$ at Cd@0.5% was read against a reference that a diverged arm had moved. On the settled reference over the declared arm set that row is **-4.2%** — readable, and negative. The $+33.9\%$ at Cd@1% is unaffected on the five-case study, and §5.1 reports what happens to it at thirteen. We report this because a protocol that only ever deleted inconvenient numbers would not be a protocol.

Rule 7, learned from the corpus: an admission threshold and a scoring threshold must be derived from the same quantity. We pre-registered a gate that admits a case if its arms agree on final drag to within 2%, and a scoring rule that can read a 1% band only if they agree to within 0.5%. Both were declared in advance, and they disagree — so the sweep admitted two cases it could not then read, and they cost us every total-drag row (§5.1). The gate should have been derived from the readability limit. We did not change it after the fact; we report the inconsistency, because a study can admit cases it cannot score and the failure is invisible until it happens.

Rule 8, from the same sweep: a converged run is finished, not truncated. OpenFOAM exits early when `residualControl` is satisfied. A scorer that judges completeness by run length alone calls that a truncation and discards it — penalising precisely the arms that converge *fastest*. Ours

did, and it silently dropped the oracle control on one case, turning a +93% control into a +49.7% bound.

Statistics. Savings 1 - warm/cold are left-skewed, so we report percentile bootstrap 95% CIs (10,000 resamples) rather than t-intervals, plus an exact two-sided sign test. At $n = 5$ the smallest attainable p is 0.0625; we state this wherever $n < 6$ rather than reporting a non-significant p as if the test could have succeeded. The thirteen-case corpus of §5.1 is where the sign test can bite: 13/13 gives $p = 0.0002$.

5. The result, and the three conditions behind it

Two studies, disjoint (§3). The **corpus** — thirteen cases, four arms — carries the headline and answers *does this generalise*. The **mechanism study** — five cases, fifteen arms — carries the controlled contrasts and answers *why, and under what conditions*. §5.1 is the first; §5.2–§5.4 are the second.

5.1 What each property of a warm start is worth

This is the paper’s central measurement, and it is a *decomposition* rather than a single number. The recommended learned seed’s +18.4% is one row of it, and the smallest.

The obvious way to run this study — hand the solver a converged field, then hand it a stored one, and compare — does not isolate anything, because the two arms differ in more than one way at a time. **oracle_mesh** carries the whole field, all four channels, at the solver’s own points; **or_proj_coarse** carries the boundary layer only, three channels, through a grid. A difference between them is not attributable to any one of those. So we added the missing control — **oracle_bl**, the exact field, all four channels, mesh-native, masked to the boundary layer — and ran the whole set on the **thirteen-case corpus**, so that every link moves exactly one property *and* every number comes from one tree at the study’s full statistical power.

Tree: corpus (13 cases, 5 arms). Row: **C_d,v@1%**, the row §4’s readability rule admits. Paired within case before averaging; percentile bootstrap 95% CI and exact sign test (`scripts/decompose.py`).

arm	field	region	delivered as	C_d,v@1%		
oracle_mesh	exact	whole	cell centres	+93.6%	[+92.9, +94.3]	13/13
or_proj_coarse	exact	boundary layer	256×64 grid, 16,384 values	+79.6%	[+72.8, +85.3]	13/13
oracle_bl	exact	boundary layer	cell centres	+72.9%	[+68.9, +76.5]	13/13
nf_bl	surrogate	boundary layer	cell centres	+18.4%	[+12.4, +25.3]	13/13
cartesian_128	exact	whole	128 ² raster, 16,384 values	+3.4%	[−2.3, +8.8]	9/13

And the differences, paired:

what moves	contrast	effect on $C_{d,v}@1\%$	cases	sign test
representation, raster	oracle_mesh $\rightarrow$ cartesian_128	-90.2 [-96.3, -84.7]	0/13 improve	$p = 0.0002$
accuracy	oracle_bl $\rightarrow$ nf_bl	-54.5 [-58.9, -49.7]	0/13 improve	$p = 0.0002$
region	oracle_mesh $\rightarrow$ oracle_bl	-20.7 [-24.2, -17.6]	0/13 improve	$p = 0.0002$
representation, body-fitted	oracle_bl $\rightarrow$ or_proj_coarse	+6.7 [-2.6, +14.8]	10/13	$p = 0.09$

What a warm start is worth, decomposed into properties that can be set independently

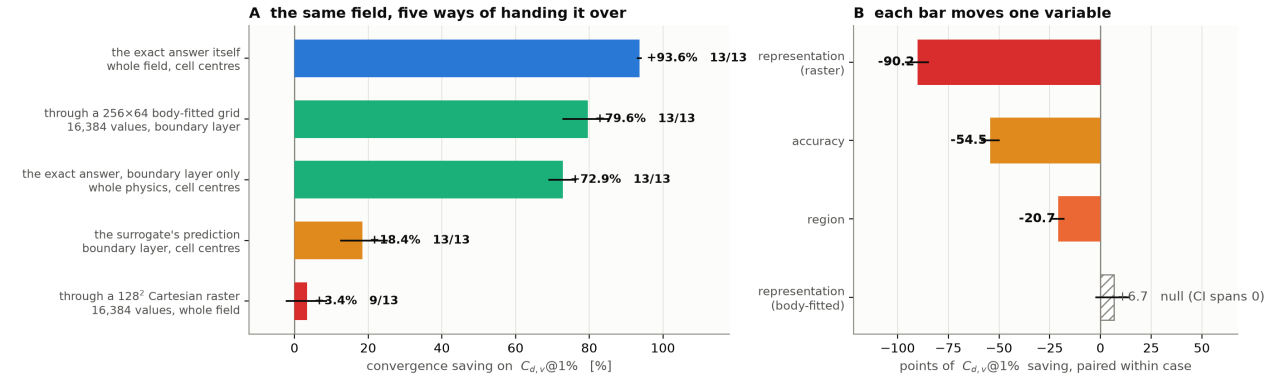


Figure 1. What a warm start is worth, decomposed. Thirteen cases, one tree, $C_{d,v}@1\%$. (A) Convergence saving for five ways of handing the solver the same field, with paired 95% bootstrap intervals and cases won. The pair to read is the exact converged field at the solver's own cell centres against the identical field stored on a 128^2 raster: +93.6% against +3.4%, an interval spanning zero. The body-fitted grid of *identical budget* sits with the mesh-native arms, which is what makes this a statement about placement rather than about grids. (B) The same data as differences, each bar moving exactly one property. A raster costs almost everything and accuracy costs more than region; the body-fitted contrast is drawn hollow because its interval spans zero — it is a null, and colouring it as a gain would overstate it.

Three readings follow.

Storage format can cost everything or nothing, depending which format. A uniform raster costs 90 of a 94-point saving, on every one of thirteen cases; a body-fitted grid holding *the same 16,384 values* costs nothing measurable. So "representation matters" is true and far too coarse. What matters is whether the format places a station where the solver keeps its state — §6's criterion, and a property of the format's grading rather than of its budget.

Accuracy matters, and matters more than region does. Replacing the exact field with the trained surrogate's prediction, changing nothing else, costs 54.5 points — more than twice what restricting the region costs, and second only to the raster. An earlier version of this paper argued "representation, not accuracy". **That is contradicted by our own control and is withdrawn.** Both matter; this table is what each is worth.

The body-fitted round trip is a null, and we report it as one because the corpus says so. On the five-case mechanism tree this contrast read +16.2 points [+12.2, +19.5], 5 of 5 — a projection making the exact field converge *better*, which we took seriously enough to look for a mechanism. At thirteen cases it is +6.7 [-2.6, +14.8], 10 of 13, $p = 0.09$: an interval spanning zero. The $n = 5$ version did not survive its own study's expansion, and we record that rather than quoting the larger number. What the row supports is the null — sending an exact field through a

body-fitted grid of this budget costs nothing measurable — and not a claim that a lossy round trip improves a seed.

We looked for the mechanism before the corpus landed, and the search is worth reporting even though the effect was not. The explanation we proposed was that a round trip low-pass filters the field, so the projected seed is smoother — in particular across the boundary-layer mask edge, where a *_bl arm hands over an abrupt transition. `scripts/mask_edge_probe.py` measures that on the seeds as the solver received them, at no compute cost, and it is **false**: the round trip makes every seed *rougher*. | | u step at the mask edge | nut step | u profile roughness | nut profile roughness | |—|—:|—:|—:|—:| | oracle_bl | 0.027 | 0.153 | 0.143 | 0.206 | | or_proj_coarse | **0.083** | **0.265** | **0.251** | **0.385** | | nf_bl | 0.055 | 0.147 | 0.142 | 0.251 | | nf_proj_coarse | **0.146** | **0.353** | **0.251** | **0.458** | The mask edge sharpens by 73–206% and the wall-normal profile roughens, for both pairs alike. It leaves a fact worth keeping: the projected seed is worse than the mesh-native one on **every** local measure this study can make — L2 error, first-cell gradient, mask-edge step, profile roughness — and converges no worse. That is the fourth independent instance here of the same pattern, **local field-quality diagnostics do not order convergence**, after §5.2.1 and §5.5 for the wall gradient and §5.5 for wall-shear smoothness.

Two things this table does not separate, both stated because they qualify the sentence readers will take away. The two representation rows are not in the same stratum. The raster contrast is `oracle_mesh` → `cartesian_128`, both whole-field; the body-fitted contrast is `oracle_bl` → `or_proj_coarse`, both boundary-layer-only. Each *link* moves exactly one property, which is what makes the −90.2 and the +6.7 valid. But the sentence "a raster costs 90 points and a body-fitted grid costs nothing" compares two links measured in different strata, and is therefore not itself a controlled contrast. It is the reading we believe, and §6 gives the quantity that predicts it, but the study does not contain the arm — a whole-field body-fitted projection — that would establish it directly. **A storage format changes several properties at once.** The raster changes sampling *position*, tangential *resolution* and interpolation together, and §6 computes only the first. Tangential resolution is nonetheless controlled, by an experiment already in this paper: §7.1's 421² raster has a tangential spacing of **0.0071 chord, finer than the wall-fitted grid's 0.0079**, and is still flat and negative. So the format that wins is the tangentially *coarser* one, and resolution in the streamwise direction is not what separates them. §5.9 rules out the remaining leading alternative — that any departure from the solver's own fixed point would do the same — for six sevenths of the effect.

5.1.1 The learned seed, and the corpus it was measured on

`scripts/corpus_probe.py`. Thirteen cases — five NACA sections, 2° to 12°, from attached flow into incipient separation — each admitted under a gate declared before the sweep ran (residual floor ≤ 1e-5, arms agreeing on final Cd to within 2%). **All thirteen were admitted**, so nothing here rests on a discretionary exclusion.

row	nf_bl	95% CI	wins	sign test	oracle control	Cartesian control
Cd_v@1%	+18.4%	[+12.4, +25.3]	13/13	p = 0.0002	+93.6% (13/13)	+3.4% (p = 0.27)
Cd_v@0.5%	+15.7%	[+10.6, +21.8]	13/13	p = 0.0002	+93.2%	+4.9% (p = 0.27)
Cd_v@0.2%	+8.8%	[+4.3, +13.5]	11/13	p = 0.022	+92.8%	-43.6% (p = 0.58)

Per case at the 1% band: **+3, +9, +9, +10, +11, +11, +16, +17, +19, +25, +28, +35, +47**. There is no losing case and no case carrying the mean.

Three properties make this a rate measurement rather than a crossing artifact: it is **monotone** across every band, the **oracle control** is flat at +93% and wins 13/13, and the **negative control** — the exact converged field resampled onto a 128² Cartesian grid — is statistically indistinguishable from no seed at all (+3.4%, p = 0.27). A pipeline that manufactured savings would not produce a null there.

Reported as a *paired* contrast rather than as two independent savings, which is the form the comparison is actually made in: the raster costs **-90.2 points [-96.3, -84.7]**, **0 of 13 cases improve**, **p = 0.0002** against the same field read at the cell centres. The other three contrasts are above, on the same tree and the same row.

Does the headline survive a stopping rule a practitioner could run? The metric above centres its band on the *converged* value, which in production nobody has. That is the right choice for measuring a mechanism and the wrong one for measuring a workflow, so `scripts/causal_stopping.py` re-scores every arm under a **causal** rule that uses only the history so far: stop at the first iteration whose trailing 50-iteration window is flat to 1%. It needs no converged answer and no cold run. | arm | the paper's metric | causal rule | |—|—:|—:| | **oracle_mesh** | +93.6% 13/13 | **+80.0%** [+77.6, +82.4] 13/13 | | **oracle_bl** | +72.9% 13/13 | **+77.8%** [+74.9, +80.8] 13/13 | | **or_proj_coarse** | +79.6% 13/13 | **+72.9%** [+69.0, +76.9] 13/13 | | **nf_bl** | **+18.4%** 13/13 | **+7.0%** [-5.1, +20.0] **6/13** | | **cartesian_128** | +3.4% 9/13 | -3.0% [-20.5, +18.1] 5/13 | **The decomposition survives the change of rule and the recommendation does not.** Every exact-field arm keeps essentially all of its advantage and all thirteen cases; §5.1's contrasts are therefore not artifacts of a converged-answer-centred band. But **nf_bl**'s +18.4% becomes **+7.0% with an interval spanning zero**. We report that rather than only the metric that flatters us, and §11 draws the consequence: what this paper contributes is the criterion and the controls, not a warm-start recommendation. **Why it changes, and it is not noise.** Under the causal rule **nf_bl** stops at 425–561 iterations on every case, almost independently of the case, while **cold** ranges 397–831. So the seed helps where the cold solve is slow and costs where it is fast, and the association is strong: Spearman **0.874** between a case's cold cost and the seed's benefit on it. Split at the median (a *post hoc* split, and labelled as one) the seven slow cases give **+26.0%** [+14.6, +36.8], **6 of 7** and the six fast cases **-15.1%** [-17.3, -12.9], **0 of 6**. That is more useful to a practitioner than the mean: **seed the solves that are expensive**.

Total drag, lift and pressure drag are unreadable over the thirteen, and two cases carry all of it. **naca4415** at 2° and 4° have arms that settle on drag values **1.04% and 1.27% apart**, against ≤0.113% for every other case in the corpus. No arm is *unsettled* on those two — they settle in different places, which is the signature of a case without a unique steady fixed point, and the same signature **naca4412@3** showed. All three are thick cambered sections at low incidence, and all three have the corpus's worst residual floors (8.75e-6 and 1.22e-6, against ~1e-7 elsewhere).

Sensitivity analysis, and it is post hoc. Dropping those two cases leaves eleven, on which Cd@1% becomes readable and reads **+30.0%** [**+5.3, +47.0**], **10/11 wins, p = 0.012**, with the oracle control at +95.4% (11/11) and the Cartesian control at -262.5%. We report it because withholding it would be hiding a result, and we label it because the exclusion was chosen after seeing the data. It is not the headline. We do **not** carry the Cd@0.5% row from that subset: the oracle control there reads -13.9% with one case at -1112%, so by our own rule the row is unreadable regardless of what `nf_bl` does.

The size of the effect, stated plainly. +18.4% is a modest acceleration, and we do not present it as more than that. What the thirteen cases establish is not a large saving but a *reliable* one: every case positive, a converged-field control at +93.6%, a negative control indistinguishable from no seed, and a monotone response across convergence bands. On a quantity that is 60–84% of the drag, obtained from a trained model on airfoils and incidences it had not seen.

The three conditions, and the study that isolates them

Everything from here to §5.6 is the **mechanism study**: `scripts/mesh_native_probe.py`, five cases, **one prediction**, one variable changed per arm, all twenty solves complete. Scored over the thirteen-arm `repr3` set with `oracle_wake` dropped (§4); including it moves no entry below by more than 0.5 points.

arm	what it hands over	residual 5e-6	Cd@1%	Cl@1%	Cd_v@1%
<code>nf_bl</code>	u, v, nut in the BL, mesh-native	< -80.5%	+33.9%	+10.1%	+14.6%
<code>nf_bl_nut</code>	eddy viscosity only	+1.2%	-293.2%	+41.1%	+42.4%
<code>nf_bl_vel</code>	velocity only	-8.2%	-40.3%	-10.3%	-4.9%

The recommended arm in this table, `nf_bl`, is the one measured at thirteen cases in §5.1. Its Cd@1% here is +33.9%, and +34.1% when re-measured in the independent tree of §5.2; §5.1 is where that number meets a corpus.

The resampled arm is not in this table, and the reason is a correction. This study originally carried a `nf_bl_proj` arm — the same prediction through a 256×64 round trip — reported at -58.8% on `C_d@1%`. That round trip took its wall-normal coordinate from the nearest surface *vertex* rather than the nearest segment, which reads this mesh’s first cell ring three orders of magnitude too far from the wall (§5.2.1). Re-measured with that fixed, the same arm reads -32.1% with a 95% interval of [-144.1, +60.9] and 3 wins in 5 — **not distinguishable from zero**. The representation contrast is therefore made in §5.2 on the *exact converged field*, where it is clean, rather than on this network prediction, where it is not.

5.2 Condition 1 — the representation must not be a uniform raster

The controlled contrast is one field handed to the solver through different representations. We make it on the **exact converged field**, so no property of any network enters, and we make it on `C_d,v` at the **thirteen-case corpus**, which is the only force row that corpus can read (§5.1).

arm (exact converged field)	representation	C _{d,v} @1%	95% CI	wins
oracle_mesh	the solver’s own cell centres	+93.6%	[+92.9, +94.3]	13/13
cartesian_128	uniform 128 ² raster, same 16,384 values	+3.4%	[−2.4, +8.7]	9/13

The same converged solution is worth 93.6% of a cold solve read at the solver’s own points and nothing at all off a raster. The raster arm’s interval spans zero ($p = 0.27$): as an initial condition it is statistically indistinguishable from uniform freestream. The intervals do not overlap and every case is paired, so this is not a spread artifact.

This is the paper’s central measurement, and three things about it matter. It uses the **exact answer**, so it cannot be explained by any surrogate’s error. It is on the **readable** row of the **thirteen-case** corpus, so it is not an artifact of a five-case tree. And the raster holds the same 16,384 values as the wall-fitted grid below, so it is not a budget effect.

A body-fitted grid is not a raster. On the same thirteen cases, the same exact field through a wall-fitted 256×64 grid — holding the *same* 16,384 values as the raster — reads **+79.6%** [**+72.8, +85.3**], **13/13** on C_{d,v}. Against the boundary-layer-matched mesh-native control that is a difference of +6.7 points with an interval spanning zero (§5.1): **no measurable cost**, where the raster costs 90.2. §6 gives the quantity that separates them, and it is neither budget nor accuracy: the raster’s first wall-normal station sits at $y^+ \sim 1725$, the wall-fitted grid’s at $y^+ \sim 37$.

What we do not claim, and why. On **total** drag the same arms read +49.7%, −278.3% and −181.6%, which looks far more dramatic — and we do not report it as a result. Every C_d row on the thirteen-case corpus is marked unreadable by §4’s rule (settled spread 1.27% against a 0.5% limit) and the converged-field control itself swings +49.7% / −42.6% / +12.8% across bands. A control that is not flat indicates a measurement that cannot be read, and an earlier draft of this paper led with those numbers. C_{d,v} is 60–84% of the drag here, it is monotone across bands, and it is the row we quote.

5.2.1 It is not the near-wall state — a result we did not expect

The obvious explanation is that a grid cannot hold the near-wall state. §6 shows that damage is real, computable and bounded. **It is not what costs the solve.** The placement ladder shows it directly: all three arms below carry the same exact field through the same round trip, and only the grading of the grid changes.

Tree: placement2 (5 cases, 9 arms). The whole ladder is reported, both families, because a ladder with half its rungs removed is not a ladder:

arm	source	values	first station	wall-gradient error	C _{d,v} @1%
oracle_mesh	exact	—	native	0.0%	+92.5%
or_proj_coarse	exact	16,384	$2.5 \cdot 10^{-4}$	1218.8%	+86.1%
or_proj_half	exact	8,192	$5 \cdot 10^{-6}$	1.8%	+71.0%
or_proj_fine	exact	16,384	$5 \cdot 10^{-6}$	1.8%	+69.0%
nf_proj_fine	network	16,384	$5 \cdot 10^{-6}$	53.1%	+17.7%
nf_proj_half	network	8,192	$5 \cdot 10^{-6}$	53.1%	+15.7%
nf_bl	network	—	native	53.7%	+14.6%
nf_proj_coarse	network	16,384	$2.5 \cdot 10^{-4}$	877.7%	+14.5%

The network family makes the same point a second time, and more cleanly. Its four arms span **16.5× in wall-gradient error** — 53.1% to 877.7% — and **3.2 points in convergence**, from +14.5% to +17.7%, all 5 of 5. Whatever the first station does to the near-wall state, it does not reach the solve. The oracle family spans a wider range in convergence (69.0% to 86.1%) and orders it *against* the gradient: the arm carrying 1218.8% error is the best of the three. Neither family supports the near-wall state as the mediator, and they fail to support it in different directions.

Placement, not budget, determines what a grid retains. Moving the first station inside the mesh’s first cell takes the wall-gradient error from 1218.8% to **1.8%** and the roughness to the converged field’s own, at half the value budget as readily as at the full one. That is §6’s criterion, confirmed.

And it does not help. The arm carrying 1218.8% gradient error is the *best* of the three exact-field arms on the readable row (+86.1% against +69.0%), and on total drag the ordering is the same. Restoring the first-cell gradient does not recover the solve; here it costs.

Three honest qualifications. The 1.8% is a single scalar — the first-cell wall-normal gradient — and not a statement about the near-wall *state*. The same arm’s boundary-layer field errors are 8.1% in *u* and **23.1% in nut**, and its *nut* damage is *worse* than the coarse arm’s 17.9%. §5.4 identifies *nut* as the least forgiving channel, so that is a live alternative explanation for the ordering, and we did not test it. Second, the grading change moves the grid’s growth ratio as well as its first station (1.141 → 1.214), so the ladder is not a strictly one-variable contrast. Third, and most limiting: §6.2 shows that at a $5 \cdot 10^{-6}$ first station the round trip is a structural **no-op** at the wall — fewer than two mesh rings lie below that station, so *clustered_seed* populates it from the first ring by nearest-neighbour donor and hands the ring back its own value. The 1.8% is therefore substantially the statement that a grid finer than the mesh reproduces the mesh, and these two arms are *not* evidence that a real surrogate emitting *u(y1)* at *y1* would carry the near-wall state. What this ladder supports is the null — restoring the first-cell gradient does not recover the solve — and not a claim about which other quantity does.

5.2.2 Where the damage is: located only weakly, and reported as such

An earlier version of this paper located the damage in the pressure field. That claim does not survive its own numbers and is **withdrawn** here.

The reasoning was: every projection preserves viscous drag while total drag collapses, so the damage must be in C_{d,p}. Three objections defeat it.

1. **It is an identity, not a measurement.** $C_d = C_{d,p} + C_{d,v}$. Showing $C_{d,v}$ preserved and C_d destroyed *entails* $C_{d,p}$ destroyed; it locates nothing. We report no field-level diagnostic of the seeded pressure, and without one the claim is arithmetic.
2. **The quoted number was censored.** `or_proj` reads -184.3% on $C_{d,p}@0.5\%$ as a bounded mean over four cases of which one never reached the band; over the three that did it is -37.8% . §4 rule 3 requires the bound *and* the reached-only value.
3. **The recommended seed has the same defect.** `nf_bl` reads -116.1% on $C_{d,p}@0.5\%$ while winning the headline row 13/13. A quantity on which the winning seed and the worst-losing seed are both strongly negative cannot be what separates them.

We also withdraw the supporting observation that smoothing the wall-law repair rescued pressure drag. On the three cases the repair arms actually reached the band, the unsmoothed repair reads $+19.8\%$ and the smoothed $+20.0\%$ — a 0.2-point difference, not the 160 points an earlier draft reported. That gap was an artifact of the arms being scored over different case sets, one censored and one dropped: precisely the failure §4 rule 3 exists to prevent, applied to our own headline. The rule caught it, and the paper is the place to say so.

What can be said. $C_{d,v}$ is preserved by every projection we ran while C_d is not, so the damage falls outside the near-wall shear and pressure is the natural suspect. **We do not have the measurement that would establish it**, and §10 records this as the paper’s principal open question rather than dressing a suspicion as a finding.

5.3 Condition 2 — hand over the boundary layer only

The region axis is controlled the same way as the resampling axis: both arms are the same network, both mesh-native, differing only in whether the handover is masked to the boundary layer. $Cd@1\%$, five cases, cold = 805 iterations, oracle control $+92.1\%$:

arm (network, mesh-native)	region handed over	$Cd@1\%$
<code>nf_bl</code>	boundary layer only	$+33.9\%$
<code>nf_mesh</code>	the whole field	$< -568.3\%$

Why it fails is not what we first wrote, and the correction is measured. The natural explanation is that the model extrapolates in the outer field — its training `sdf` distribution is centred on 0.23 chords while the C-grid reaches 20. That explanation is **falsified by §5.7**: grid sequencing hands over a *coarse mesh solution*, which extrapolates nowhere, and the boundary-layer-restricted arm still beats the whole-field one on every readable row ($+75.9\%$ against $+63.1\%$ on $C_{d,v}$, -302.4% against -375.4% on C_d). So the restriction is not about the surrogate’s trust region.

The next explanation — that a mapped whole field is not divergence-free on the fine mesh while a freestream outer field is — is falsified too, from the solver’s own first continuity error: every seed but the converged-field oracle sits at $\sim 1.4 \cdot 10^{-6}$, and the whole-field arm is if anything the *cleanest* of them at $1.19 \cdot 10^{-6}$ while performing worst. Divergence does not discriminate.

What survives is the consistency reading of §5.4: a seeded region that is inconsistent with the state the solver holds elsewhere leaves a smooth, global mismatch for the pressure correction to remove, and the larger that region the more there is to be inconsistent about. We state this as the surviving candidate, not as an established mechanism, and §5.2.2 records what happened to the previous candidate when we tried to establish it.

Together with §5.2 this gives two controlled contrasts sharing a common arm (`nf_bl`), one per axis, each changing a single variable. Neither factor alone is sufficient: mesh-native evaluation of the whole field is the worst arm in the study, and boundary-layer restriction under resampling is still negative.

We deliberately do **not** present these as a 2×2 . The fourth corner — a network prediction of the whole field, resampled — was never run, and the arms that would fill it come from a different population (oracle rather than network). A table whose cells mix provenance would read as a design when it is a gap.

The oracle bound on the resampled row, which removes the accuracy confound entirely, is reported separately and is the stronger statement:

arm	field	representation	Cd@1%
<code>fitted_256x64</code>	exact converged answer	wall-fitted 256x64	-172.6%
<code>fitted_bl</code>	exact converged answer, BL only	wall-fitted 256x64	-206.1%
<code>cartesian_128</code>	exact converged answer	Cartesian 128 ²	-548.4%

Even a perfect field, resampled, costs the solver more than starting it cold.

5.4 Condition 3 — velocity and eddy viscosity together

This condition was not anticipated; it came out of splitting the channels.

Eddy viscosity alone is simultaneously the **best** arm in the study on viscous drag (+42.4%) and lift (+41.1%), matching the oracle projection, and the **worst** on total drag (-293.2%). Velocity alone is bad at everything. Only the pair is positive on total drag.

The reason is the structure of the Spalart–Allmaras production term, which is driven by the strain rate. An eddy viscosity handed over without the velocity field that generated it is inconsistent with the strain the solver computes from the field it actually has; the momentum sink is therefore wrong, the pressure field must reorganise, and `Cd_p` — 16–40% of the drag here — is destroyed. The shear-driven quantities do not care, and speed up.

Consistency between handed-over channels is not a detail of the recipe. It is most of it, and §7.3 finds the same lesson again at a completely different length scale.

5.5 Restoring the wall gradient does not recover the solve

Section 6 shows the damage a projection does is available in closed form. A factor that is known can be divided back out, so we built the repair the mechanism implies: invert the law of the wall at the representation’s own first station to recover `u_tau`, using **only what the representation already carries**, and re-evaluate the profile at each cell’s own wall distance (`solver/placement.py`).

It works on the gradient, and then on its smoothness too. The first repair restored the magnitude but left the reconstruction rough along the surface, because every station is inverted independently from its own value. Smoothing the recovered `u_tau` over an arclength window fixes that. Measured on the seeds exactly as the solver received them, five cases:

Tree: repair (5 cases, 8 arms).

arm	first-cell gradient error	roughness ($\times$ converged)
nf_bl (mesh-native, the seed that works)	53.7%	4.91 $\times$
nf_proj	877.7%	22.95 $\times$
nf_proj_fix (repaired)	69.4%	29.50 $\times$
nf_proj_smooth (repaired + smoothed)	73.0%	6.36$\times$
or_proj	1218.8%	18.68 $\times$
or_proj_fix	54.4%	24.10 $\times$

nf_proj_smooth matches the working seed on **both** diagnostics this study can measure: 73.0% gradient error against its 53.7%, and 6.36 $\times$ roughness against its 4.91 $\times$, where the unrepaired projection is at 877.7% and 22.95 $\times$. Note that the repair *alone* makes the wall shear rougher, not smoother — 29.50 $\times$ against the projection’s 22.95 $\times$ — because each station is inverted independently; smoothing is what brings it back.

And the solve does not care.

arm	C_d@1%	C_d,v@1%
nf_bl (mesh-native)	+34.3%	+14.6%
nf_proj	−32.1%	+14.5%
nf_proj_fix	−32.0%	+11.7%
nf_proj_smooth	−31.4%	+11.5%
or_proj	−182.1%	+86.1%
or_proj_fix	−180.7%	+56.7%

Three seeds spanning **877.7% to 69.4%** in gradient error and **29.50 $\times$ to 6.36 $\times$** in roughness all land between −31.4% and −32.1% on total drag, where the mesh-native seed reads +34.3%. On viscous drag the repair is not merely neutral but slightly harmful. And **nf_proj**, carrying sixteen times the mesh-native seed’s gradient error, already matches it on viscous drag (+14.5% against +14.6%).

We report this as a falsification, because that is what it is. We predicted the repair would work, registered the reasoning before running it (`docs/protocols/placement_prediction.md`), built it, found the one measurable difference that remained, removed that too, and it still did not work. §6.7 states what that costs the paper’s mechanism, which is a great deal.

One secondary observation, flagged rather than promoted. On *pressure* drag — which converges three times slower than total drag here and whose 1% row is unreadable — the smoothed repair is the only seed in this study that is positive: **+19.8%** at the 0.5% band and **+57.8%** at 0.2%, against −115.4% for the recommended seed. A rough wall-shear distribution driving a spurious pressure response is a coherent reading, and it fits §5.4’s finding that pressure is where inconsistent seeds do their damage. It was found after the fact, on a secondary quantity, so §4’s protocol says it is an observation and not a result. The experiment that would settle it is a smoothed *mesh-native* seed, which this study does not contain.

5.6 The residual objection

nf_bl is **negative on the residual at every depth** and positive on drag; the projected arms are the exact inverse. A reviewer is right to look hard at that. Our answer has three parts.

1. **The residual is not the objective.** Nobody runs a RANS solve to obtain a small residual; they run it to obtain a force coefficient that has stopped moving. The residual is a proxy, and this study measures the proxy failing — on the *exact converged field*, so the disagreement cannot be blamed on our network:

arm (exact field)	residual $5 \cdot 10^{-6}$		C_d@1%	wins
or_proj_coarse	+40.6%	−181.6% [−318, −45]		1/5
or_proj_fine	+36.6%	−266.8% [−419, −114]		0/5
nf_bl (recom- mended)	−80.5%	+34.1% [+14, +53]		4/5

The two arms that look best on the residual are the two worst on drag, and the arm we recommend is the worst on the residual. **A study that scored the residual alone would have selected exactly the wrong seed**, and would have done so with intervals excluding zero. We report both metrics, always.

1. **We know what the residual is rewarding.** The Ux residual measures how much the field changes per iteration, so it rewards smoothness. A resampled field has had its near-wall structure interpolated away and therefore changes less, while carrying 877.7% error in the wall gradient. The residual is not being fooled at random — it is faithfully measuring something that is not drag.
2. **The choice of metric was pre-committed and cuts against us.** The force metric replaced the residual metric for the reason in §4 rule 1, which predates this arm; and the readability rule then rejected rows we would rather have quoted, including the withdrawal above.

The honest residue stays in the paper: **on the residual, nf_bl is worse than a cold start.** §8 is what makes that survivable — the same 25-iteration gate that bounds drag bounds the residual metric’s worst case at −7.6%.

5.7 The classical baseline beats the learned seed on both axes

The comparator that matters is not a uniform freestream. Production aerodynamics warm starts by **grid sequencing** — solve on a coarsened mesh, map the result up, continue on the fine one — and a learned initialisation measured only against a cold start has not answered the question a practitioner asks. We run it: the same C-grid family coarsened by two (7,850 cells against 31,700, first cell $2 \cdot 10^{-5}$ against 10^{-5}), mapped with a nearest-cell map in body-fitted coordinates, and its coarse solve **charged**.

row	cold	oracle	nf_bl	sequenced_bl	sequenced_vel	sequenced_nut	sequenced
C_d,v@1%	696	+92.4%	+14.6%	+75.9%	+2.2%	+34.9%	+63.1%
C_d@1%	802	+92.1%	+33.9%	−302.4%	−4.2%	—	−375.4%
C_l@1%	944	+99.9%	+10.1%	+49.4%	−0.1%	−59.7%	+33.1%

It makes a far better seed. On the readable row grid sequencing reads +75.9% against our +14.6% — five times the saving — which is what §6 predicts for it: a coarsened body-fitted mesh keeps its wall-normal stations clustered at the wall, so it satisfies the placement criterion by construction. **The criterion correctly predicts the behaviour of a method containing no network, no training data and no surrogate, and from which it was not derived.**

And it is also cheaper, which an earlier draft of this paper got wrong. The coarse solve is a real cost and must be charged. We first charged it at its full 6000-iteration budget — 1486 fine-mesh-equivalent iterations at the cell-count ratio — and concluded that grid sequencing lost on price. That is not what a practitioner does. A coarse level is run until its own forces settle. Charging it at the iteration where the *coarse* solve’s $C_{d,v}$ enters and stays in its own 1% band:

case	coarse iterations to its own 1% band	fine-equivalent charge
naca0012@0°	186	46
naca0012@4°	206	51
naca0015@6°	246	61
naca2412@2°	193	48
naca2415@5°	240	59
mean	214	53

The charge is **53 fine-equivalent iterations, not 1486**. `sequenced_b1` then reaches the band at ≈ 168 fine iterations, so the total is ≈ 221 against a cold 696 — a **$\approx 68\%$ end-to-end saving**, against the learned seed’s +14.6%.

We therefore withdraw the claim that the learned seed’s advantage is price. On this configuration classical grid sequencing is better on both axes, by a factor of roughly four. Two caveats both point the same way — the mapper is ours rather than a production `mapFields`, and §5.7’s own diagnostic puts it at a 6–10 $\times$ first-cell gradient overestimate where the coarse mesh’s placement would permit about 2 $\times$, so this is a **lower bound** on grid sequencing.

What remains for the learned seed is a workflow argument, not a performance one: it needs no second mesh, no second solver configuration and no coarse solve, and it produces its seed in ~ 11 s of inference. That is worth something in an automated design loop. It is not a speed record, and §11 says so.

The one thing grid sequencing does not fix. Both `sequenced` arms are strongly negative on *total drag* (-302% , -375%), as our own projections are. The channel split is the same as §5.4’s: the saving rides on `nut` (+34.9% alone) while velocity alone is inert (+2.2%). So the classical method inherits the same total-drag pathology, which is further evidence that it belongs to the solver and the metric rather than to any particular seed.

5.8 A common-mode limitation, checked rather than assumed

`write_case` floors `nuTilda` at its freestream value, which clips 37% of the boundary-layer cells. That sounds like it could produce these swings. It cannot: the clipped cells hold values $\sim 88\times$ below the peak, so the floor removes only **2.1–2.3%** of the eddy-viscosity field’s energy, and it applies identically to every arm including the oracle. It is a common-mode limitation of the study, not a differential effect that could move an arm from +33.9% to -293.2%.

5.9 Is it the representation, or any departure from the solver’s fixed point?

This is the strongest objection to §5.2 and it needs answering directly. The oracle seed is the cold run’s own converged solution re-injected: to solver tolerance it is a **fixed point of the discrete**

operator, so it converges in ~ 52 iterations because its residual is already at the floor, not because it is physically excellent. Every other arm is a perturbation of that fixed point, and any perturbation restarts a transient. Read that way, "storing the field on a raster is catastrophic" and "moving away from the discrete fixed point at all is catastrophic" fit §5.2 equally well — and the second is a much weaker paper.

The control (`scripts/perturbation_probe.py`, tree `perturb`, 5 cases). Perturb the exact converged field by a *smooth* field carrying the **same per-channel L2 norm** as the 128^2 raster round trip's error, and hand it over mesh-native. Two properties make it a control rather than another bad seed:

- it is six Fourier modes whose shortest wavelength is a third of a chord against a boundary layer 0.0187 chords thick, so it carries no structure on the scale the near-wall state lives at; and
- it is **ramped to zero inside the boundary layer** by a smoothstep, so the near-wall field the solver receives is the converged one exactly.

Measured on the seeds as the solver received them, the two arms carry the identical whole-field error and could not differ more in where it sits:

seed	whole-field u error	u error in the layer	u error in the first ring
<code>cartesian_128</code>	21.51%	45.46%	1974.90%
<code>smooth_perturb</code>	21.51%	0.00%	0.00%

The result, $C_{d,v}$, paired within case, five cases, every band readable (settled spread 0.076% against a 0.5% limit):

band	<code>oracle_mesh</code>	<code>smooth_perturb</code> , paired	<code>cartesian_128</code> , paired
$C_{d,v}@1\%$	+92.4%	− 12.2 [−15.2, −8.0]	− 82.5 [−89.0, −76.1]
$C_{d,v}@0.5\%$	+91.9%	− 129.7 [−183.9, −71.0]	−79.6 [−86.5, −72.7]
$C_{d,v}@0.2\%$	+91.3%	− 237.5 [−313.2, −162.9]	−139.4 [−201.1, −80.2]

Two findings, and the second was not what we went looking for.

At the band this paper reports, error magnitude is not what costs. The same whole-field error norm costs 12.2 points spread smoothly and 82.5 delivered by a raster — 6.8 times more, with intervals that do not overlap. So the raster's harm at $C_{d,v}@1\%$ is **not** attributable to mere distance from the discrete fixed point, and roughly six sevenths of it is attributable to *where* the error sits. That is what §5.2 claims and what §6 computes, and it is the objection answered.

But a smooth outer error is not cheap — it is deferred, and the crossover is the mechanism §6.7 could only argue for. `smooth_perturb` is far better than the raster at a 1% band and far *worse* at 0.5% and 0.2%. That is exactly what a SIMPLE-family solver should do. Its near-wall field is exact, so $C_{d,v}$ — a near-wall integral — enters a loose band almost at once; and its error is a smooth, globally elliptic mode, which is the one the pressure correction removes slowest, so it never clears. The raster's error is the opposite: high-wavenumber and near-wall, punished immediately by the loose band and then largely annihilated by a few under-relaxed momentum sweeps, which is why its penalty grows much more slowly with depth.

This is the first direct measurement in this paper of the smoothing argument. §6.7 offers it as an interpretation that fits four otherwise-unrelated observations; here it makes a falsifiable

prediction — smooth error should be cheap early and expensive late, high-wavenumber error the reverse — and the prediction holds on a controlled pair. We still do not claim it as the mechanism of §5.2, because this arm perturbs a field rather than decomposing one, and because a single crossover on five cases is evidence and not proof.

Reported against our own rule. §4 says a saving that is not monotone across bands is not a clean rate measurement. `smooth_perturb` is emphatically not monotone ($+80.2\% \rightarrow -37.8\% \rightarrow -146.2\%$ in absolute saving), so its $+80.2\%$ at the 1% band must be read as "it enters this band quickly", not as "it converges quickly". The contrast that survives that caveat is the *comparison* at a fixed band against `cartesian_128`, which is what the first finding above rests on and what the objection required.

What this arm cannot show. It cannot show that distance from the fixed point is irrelevant: `smooth_perturb` costs a real 12.2 points at 1% and far more at depth, so departing from the solver's own fixed point is not free, and any study using a converged-field oracle should expect that floor. Nor could it have shown otherwise — the perturbation is zero exactly where the raster does its worst damage, by construction. The claim available is the one made above, and we do not make the stronger one.

A second arm at twice the norm (`smooth_perturb_x2`) was run to check that a null was not simply a perturbation too small to matter. It is not reported here: the first finding is a positive contrast rather than a null, so the guard is not needed for it, and the arm's own rows are censored on this budget.

6. The mechanism, in closed form

Everything in §5 follows from one quantity, and that quantity can be computed before any solve is run, from two numbers a CFD engineer already has.

6.1 What a projection actually does to the near-wall state

Viscous drag is the surface integral of the tangential velocity gradient in the first cell off the wall. On the meshes this paper uses that cell centre sits $5 \cdot 10^{-6}$ chords from the surface, and viscous drag is 60–84% of the total.

When a field is resampled through a grid, every mesh cell nearer the wall than the grid's first wall-normal station receives the value belonging to that station. There is nothing else for it to receive: the representation holds no sample in between. The reconstructed first-cell gradient is therefore not *degraded* — it is replaced by a different quantity, the velocity at the station divided by the distance to the cell.

Write `h1` for the representation's first wall-normal station, `y_c` for the mesh's first cell centre, and `u(y)` for the true near-wall profile. The seeded first-cell gradient is `u(h1)/y_c` where the true one is `u(y_c)/y_c`, so the gradient is overestimated by

$$G = u(h_1) / u(y_c)$$

and in wall units this is a statement about the law of the wall alone:

$$G = u^+(y_1^+) / u^+(y_c^+), y^+ = y \ u_tau / \nu$$

with $u^+ = y^+$ in the viscous sublayer and $u^+ = \ln(y+)/\kappa + B$ in the log layer. **There is no fitted parameter in this expression.** u_tau comes from the converged solution's own wall gradient and ν from the case.

6.2 It is a parameter-free upper bound on the damage

Table 2 tests the closed form against the measured first-cell gradient of a wall-fitted 256×64 projection of the **exact converged field**, over five cases, at five first-station heights spanning a factor of fifty. u_tau is taken from each case's own converged wall gradient and y_c from that case's own first cell centre ($3.79\text{--}4.01 \cdot 10^{-6}$ chords, $y^+ = 0.56\text{--}0.59$; the C-grid's *nominal* first cell centre, half of $\text{first_cell} = 10^{-5}$, is $5 \cdot 10^{-6}$, and this paper quotes the measured value throughout). Nothing is tuned. Produced by `scripts/validate_closed_form.py`, which runs no solver.

first station	y^+	mesh rings below it	predicted G	measured G	predicted/measured
$2.5 \cdot 10^{-4}$	36.8	11	$24.2 \times$	$12.7 \times$	1.91 ± 0.07
$1.0 \cdot 10^{-4}$	14.7	7	$17.5 \times$	$7.8 \times$	2.26 ± 0.13
$2.5 \cdot 10^{-5}$	3.7	2	$6.45 \times$	$2.34 \times$	2.77 ± 0.18
$1.0 \cdot 10^{-5}$	1.5	1	$2.58 \times$	$1.00 \times$	<i>no-op</i>
$5.0 \cdot 10^{-6}$	0.7	1	$1.29 \times$	$1.00 \times$	<i>no-op</i>

The bound is claimed over the top three rows, and it holds there: the expression over-predicts by $1.9 \times$ to $2.8 \times$. We report it as what it is — a *parameter-free upper bound*, correct in direction and in ordering, never optimistic. It is not a 13%-accurate estimate, and an earlier version of this work claimed that it was; that figure came from a projection whose wall-normal coordinate was measured to the nearest surface *vertex* rather than the nearest segment, mis-placing every near-wall cell by three orders of magnitude. The bug is fixed, the number is corrected, and the correction is recorded here rather than quietly absorbed.

The bottom two rows are excluded, and the reason is structural rather than statistical. They read $G = 1.000000$ with a standard deviation of 10^{-16} across five different airfoils, and a measurement with no variance across five geometries is an identity, not an agreement. The cause is in the third column. `clustered_seed` populates each of its own stations by nearest-neighbour donor *from the mesh*; when fewer than two mesh rings lie below the representation's first station, that station is populated from the first ring and mapped straight back to it, so the round trip is a no-op at the wall and cannot exercise the clipping §6.1 models. The count is computable from the mesh alone, before any measurement, so those rows are marked and dropped by criterion rather than because their answer was inconvenient.

Two consequences we are obliged to draw from that, both against us. First, the over-prediction in the live rows is partly the same effect in milder form: the implementation clips a query below its first station rather than extrapolating, so a bound is the honest reading of an expression that assumes the worst about an implementation detail. Second, and more important, §5.2.1's " $1218.8\% \rightarrow 1.8\%$ " is **substantially the statement that a grid finer than the mesh reproduces the mesh**, not evidence that a graded surrogate output would carry the near-wall state — a real surrogate

emits $u(y_1)$ at y_1 , and its own value there, rather than the donor cell's. §10 states what this does and does not licence.

What the bound is for. Ruling a format out. At $y^+ = 37$ it says "expect order $20\times$ ", and $12.7\times$ is measured; at $y^+ < 1$ it says "expect nothing", and nothing is what happens. Between those, it orders representations correctly without a solve, which is the decision it exists to support.

Where the law of the wall is doing the work, and where it is not. The u^+ cap of §6.1 — no station can carry more than freestream — binds for every uniform-raster format in Table 3, whose first stations sit at $y^+ = 430\text{--}1725$ — out in the wake region of the layer, at 19% to 63% of its thickness, where the log law extrapolated that far returns a velocity above freestream. Where the cap binds, G reduces to $(u_{\text{inf}}/u_{\text{tau}})/u^+(y_c)$, an expression containing no κ , no B and no logarithm: it says only *"the seed puts freestream velocity in the first cell"*. The law-of-the-wall content of the criterion is therefore active in the wall-fitted rows at $y^+ \approx 4\text{--}37$ and **vacuous in exactly the rows that carry the headline**. This does not change any number — the cap is what makes those rows a bound rather than a wild over-prediction — but "this is the law of the wall and nothing else" is a claim about the wall-fitted regime only, and we do not make it about the rasters.

6.3 Two consequences that decide how a surrogate should be built

Refining the raster cannot fix this, and the formula says why. u^+ grows *logarithmically* in y^+ . Going from a 128^2 to a 512^2 raster spends sixteen times the values and moves h_1 by a factor of four, which moves $u^+(y_1)$ by $\ln(4)/\kappa \sim 3.4$ on a value of ~ 23 — about 15%. That is the closed-form version of the measured resolution ladder in §7.1, which is flat from 128^2 to 421^2 , and of the estimate that one cell across the inner layer would need $N \approx 11,800$, some $28\times$ beyond what the standard datasets hold.

Placement, not budget, is what a representation's retention depends on. The alternative to sixteen times the values is to move the first station inside the first cell. A 64-level geometric stack from $5\cdot 10^{-6}$ to 1 chord has a growth ratio of 1.214; a 32-level stack has 1.483. Both are ordinary meshes, and the second holds *half* the values of the 256×64 grid that loses the gradient.

Measured directly, over a **fourfold** cut in budget and at the stations where the round trip is not a no-op:

first station	16,384 values	8,192 values	4,096 values
$2.5\cdot 10^{-4}$	$12.666\times$	$12.666\times$	$12.666\times$
$1.0\cdot 10^{-4}$	$7.785\times$	$7.785\times$	$7.785\times$
$2.5\cdot 10^{-5}$	$2.338\times$	$2.338\times$	$2.338\times$

Cutting the budget by four moves the damage by a factor of 1.00 — the columns agree to every digit printed — while moving the station down the same table moves it $5.4\times$, and $12.7\times$ against a representation that resolves the first cell. The reason the columns are *identical* rather than merely close is the point: n_n sets how many wall-normal stations there are, not where the first one sits, and only the first one is in front of the mesh's first cell.

This measurement replaces an earlier one that could not have shown an effect. The five-case solve tree tests budget by comparing `or_proj_fine` (16,384 values) against `or_proj_half`

(8,192) — but both place their first station at $5 \cdot 10^{-6}$, which Table 2 marks as a no-op, so the comparison was between two round trips that were already the identity at the wall. The conclusion survives; the test that had been offered for it did not.

This is a statement about retention, and only about retention. §5.2.1 also shows that the correctly-graded grid **converges worse** than the badly-graded one. Getting the near-wall state right is not what makes a warm start work, and nothing in §6 should be read as advice to grade a surrogate’s output for speed. What §6 buys is a cheap, conservative way to know what a format keeps — which is worth having, and is not the same thing.

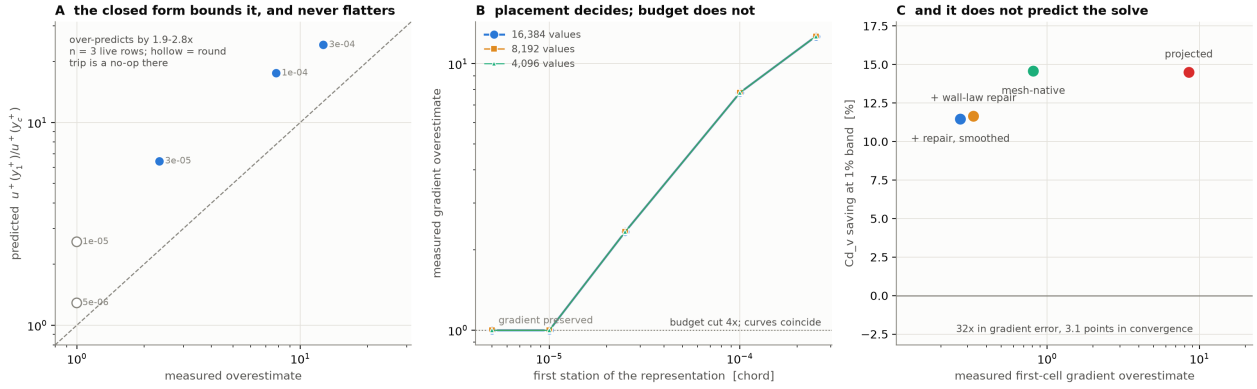


Figure 3. The criterion, what it controls, and what it fails to predict. (A) Predicted first-cell gradient overestimate against the measured one, at five first-station heights spanning a factor of fifty, with the identity line drawn. Every point sits above it — the expression is an upper bound and never flatters. The two hollow points are the rows where the round trip is a structural no-op (§6.2) and carry no information about the mechanism; the bound of 1.9–2.8× is claimed over the three filled ones. (B) The same measured damage against the first station’s height, one line per value budget. Cutting the budget fourfold, from 16,384 stored values to 4,096, leaves the three curves lying exactly on top of one another, while moving the station along the axis moves the damage 12.7×. Placement, not budget. (C) And none of it predicts the solve. Measured gradient error against $C_{d,v}$ convergence saving, one point per arm: 32× in gradient error, 3.1 points in convergence. This panel is the paper’s negative result drawn, and §6.7 is what it means.

6.4 The pre-flight check

The criterion is therefore executable. Given a target mesh’s first cell height and the wall-normal shape of the format a surrogate would emit, **G** follows immediately, and with it a verdict: a representation with no station inside the first cell has no sample of the state that viscous drag integrates, and will misreport it by roughly **G**. We ship this as `neuroforge.solver.placement` and as a command-line tool, so that the check can be run on a mesh and a format that have nothing to do with this study:

```
“ python scripts/preflight.py -first-cell 1e-5 -re 3e6 -fitted 256x64@2.5e-4 ... predicted
wall-gradient overestimate 19.18x [wall_law] FAILS. Expect the first-cell wall gradient
to be overestimated by about 19.2x. “
```

The criterion is necessary, not sufficient, and the paper is careful about this. Every representation that loses the gradient costs the solve, without exception across every arm measured here — so the check rules formats *out* for free. It does not rule them in, and we have two independent

demonstrations of that. `nf_mesh` retains the gradient perfectly and is the worst arm in the study. And §5.5 restores the gradient of a failing representation to *better than mesh-native* and the solve still does not recover. Conditions 2 and 3 exist for this reason, and the pre-flight check should be read as a veto, never as an endorsement.

6.5 The criterion applied to the formats the field actually ships

Table 3 evaluates the closed form for the output formats a surrogate might emit, against the mesh used throughout this paper. It costs no solve and no network, and it is the whole argument in one place.

Both this table and §7.1's ladder are generated by `scripts/criterion_tables.py` from **one** stated triple — `u_tau` = 0.0491 and `y_c` = $3.88 \cdot 10^{-6}$ (`y_c+` = 0.571), each the mean over the five cases' own measured values, and `nu` = $3.33 \cdot 10^{-7}$. They were previously computed separately and disagreed, quoting the same 128^2 raster at $29.3\times$ here and $36.6\times$ in §7.1 because they had been evaluated at `u_tau` = 0.0477 and 0.0427 respectively, neither of which is what this study measures. Every raster's first station is half its cell, where a cell-centred rasteriser puts its nearest sample to the wall. The verdict thresholds are fixed in that script: **passes** at $G \leq 1.5$ — inside the bound's own $1.9\text{--}2.8\times$ over-prediction, so consistent with no measurable damage, and measured at $1.00\times$ where §6.2 measures it — *degraded* to $10\times$, *fails* above.

output format	values	first station	y+	predicted G	regime	verdict
uniform raster 128^2 , 3-chord crop	16,384	$1.17 \cdot 10^{-2}$	1725	$35.7\times$	saturated	fails (bound)
uniform raster 256^2 , 3-chord crop	65,536	$5.86 \cdot 10^{-3}$	863	$35.7\times$	saturated	fails (bound)
uniform raster 512^2, 3-chord crop	262,144	$2.93 \cdot 10^{-3}$	431	$34.7\times$	wall law	fails
uniform raster 128^2 , 1-chord crop	16,384	$3.91 \cdot 10^{-3}$	575	$35.7\times$	saturated	fails (bound)
wall-fitted 256×64 from $2.5 \cdot 10^{-4}$	16,384	$2.5 \cdot 10^{-4}$	37	$24.2\times$	wall law	fails
wall-fitted 256×64 from $2.5 \cdot 10^{-5}$	16,384	$2.5 \cdot 10^{-5}$	3.7	$6.4\times$	wall law	degraded
wall-fitted 256×64 from $5 \cdot 10^{-6}$	16,384	$5.0 \cdot 10^{-6}$	0.74	$1.3\times$	wall law	passes
wall-fitted 256×32 from $5 \cdot 10^{-6}$	8,192	$5.0 \cdot 10^{-6}$	0.74	$1.3\times$	wall law	passes
mesh-native, queried at cell centres	native	$3.88 \cdot 10^{-6}$	0.57	$1.0\times$	resolved	passes

Two rows carry the paper. A **512^2 raster holds 262,144 values and still fails**, at $34.7\times$; a **wall-fitted grid of 8,192 values — one thirty-second of that budget — passes**. Sixteen times the values cannot buy what one grading decision gives away for free.

The first four rows all read $35.7\times$ or $34.7\times$ and three of them are marked *saturated*: there the `u+` cap binds and `G` reduces to $(u_{\text{inf}}/u_{\text{tau}})/u+(y_{\text{c}}+)$, which contains no `kappa`, no `B` and no logarithm. For those rows the criterion is making the weaker statement "*the seed puts freestream velocity in the first cell*" — true, sufficient, and not a law-of-the-wall result. §6.2 states this limit; it is why those

rows are labelled bounds.

The practical reading is a design rule, not a ranking of architectures. Any surrogate whose output is a uniform raster over a crop of order the chord cannot warm-start a wall-resolved RANS mesh, however finely it is rasterised, because a uniform grid must resolve its smallest scale everywhere and the near-wall scale collapses like ν/u_τ . Surrogates that predict on native mesh points satisfy the criterion by construction. Between the two sits a large and mostly unexplored middle — graded wall-fitted outputs — where the criterion is satisfied or not purely by the choice of first station. §5.2 measures that middle directly, and §5.5 shows that reconstructing the missing near-wall profile after the fact is **not** an adequate substitute for placing a station there in the first place.

6.6 The criterion across Reynolds number, at no compute cost

Because the closed form is written in wall units it makes a prediction about a regime this study never set out to measure, and it is not the obvious one:

On the same mesh, a lower Reynolds number makes the projection worse.

Both y^+ values fall together as ν rises, but through different parts of the profile. The mesh’s first cell sinks deeper into the *linear* sublayer, where $u^+ = y^+$ falls in proportion; the representation’s fixed station at $2.5 \cdot 10^{-4}$ remains in the buffer or log layer, where u^+ falls only logarithmically. The ratio — the damage — therefore grows.

This is testable with no new solves. Converged cold solves on the *same* C-grid already exist at $Re = 10^3$ to $3 \cdot 10^6$. Projecting each through the same wall-fitted grid gives Table 4 (`scripts/reynolds_transfer.py`):

Re	y^+ first cell	y^+ station	predicted G	measured G	pred/meas	weak-shear surface
10^3	0.001	0.1	$65.9\times$	$21.7\times$	3.04	60%
10^4	0.004	0.3	$65.9\times$	$21.5\times$	3.08	62%
10^5	0.027	1.8	$65.9\times$	$21.1\times$	3.13	42%
10^6	0.21	13.6	$46.7\times$	$19.1\times$	2.45	11%
$3 \cdot 10^6$	0.56	37.1	$24.5\times$	$13.0\times$	1.89	6%

Every row is measured at each case’s own first cell centre and aggregated as a *ratio of surface integrals*, matching §6.2 exactly — the bottom row’s $24.5\times$ and $13.0\times$ are Table 2’s $2.5 \cdot 10^{-4}$ row ($24.2\times$, $12.7\times$) measured on two of the same cases, which is the consistency check this table exists to survive. An earlier version probed at a fixed $4 \cdot 10^{-6}$ and averaged *pointwise ratios*, which reads high because stations near stagnation carry a vanishing true gradient and dominate an average of ratios while contributing nothing to the integral; it reported $15.0\times$ here against $12.7\times$ there for the same configuration.

The direction holds across three and a half decades: the same representation on the same mesh costs $13.0\times$ at $Re = 3 \cdot 10^6$ and $21.7\times$ at $Re = 10^3$. A practitioner’s instinct — that a coarse representation is more forgiving at low Reynolds number, where the flow is smoother — is the wrong way round, and the reason is that the mesh’s first cell has moved into the linear sublayer while the representation’s has not.

The effect is real but modest, and it saturates. The measured damage rises by about 67% over that range rather than by the factor of 2.7 the unbounded expression suggests, and it is flat below $\text{Re} = 10^5$. The bound loosens in the same direction: predicted/measured grows from 1.9 at $\text{Re} = 3 \cdot 10^6$ to 3.0 at $\text{Re} = 10^3$. The last column says why. It reports the fraction of surface stations carrying under a tenth of the peak wall shear — the signature of a laminar or separated layer. At $\text{Re} \geq 10^6$ it is 6–11%; at $\text{Re} \leq 10^5$ it is 42–62%, the boundary layer is laminar and largely separated, and the law of the wall does not describe it at all. The expression is then a bound in the loosest sense: it still gets the sign and the ordering right, and it still errs conservatively.

These numbers were re-measured on 2026-09-01 after the wall-distance defect described in §5.2 was fixed. The earlier version of this table read $24.7\times$ at $\text{Re} = 3 \cdot 10^6$ rising to $179\times$ at $\text{Re} = 10^3$, a far more dramatic trend, and it was an artifact of that defect. The direction survived the correction; the magnitude did not, and the corrected magnitude is what is reported.

6.7 What the criterion is for, and what it is not

The sections above establish what a representation does to the near-wall state, and they establish it firmly: the damage is computable, bounded above, ordered correctly across a fifty-fold range of first stations and three and a half decades of Reynolds number, and controlled by station placement rather than by value budget. **It does not follow that this damage is what makes a seed slow, and §5.2.1 shows directly that it is not.**

Three independent attempts to make the near-wall state the mediator all fail:

1. **By placement** (§5.2.1). Moving the first station inside the mesh’s first cell takes the wall-gradient error from 1218.8% to 1.8% and the roughness to the converged field’s own. Convergence gets worse: -266.8% against -181.6% , winning 0 of 5 cases against 1 of 5.
2. **By repair** (§5.5). Inverting a wall function restores a projected seed’s gradient from 877.7% to 69.4%. Convergence moves 0.1 points.
3. **By smoothing that repair** (§5.5). Bringing the reconstruction’s roughness to $6.36\times$ converged, against the working seed’s $4.91\times$, moves convergence a further 0.6 points.

A seed can reproduce the converged boundary layer to 1.8% in gradient and exactly in roughness and still be among the worst initial conditions measured here. **The near-wall velocity field is not the mediator.**

Where the damage is instead (§5.2.2). Every projection preserves *viscous* drag — the quantity the wall gradient integrates, and 60–84% of the drag — at +69% to +86% against the mesh-native oracle’s +92.5%, all 5 of 5 cases. What it costs is total drag, and therefore pressure drag. We can say that much and no more: on this five-case tree the **C_{d,p}** rows are censored — between one and three arms never reach the band inside the budget — and §4 rule 3 forbids reading a mean across arms scored on different case sets. On the three cases that *do* reach, the arms we had proposed as a contrast read +19.8% and +20.0%, which is no contrast at all. **An earlier version of this paper built a pressure mechanism on those rows and it is withdrawn;** §5.2.2 records what it was and why it does not stand.

One reading fits everything here, and we offer it as a reading. In a SIMPLE-family solver, error in the near-wall velocity is high-wavenumber and is annihilated by a few under-relaxed momentum sweeps, whereas an error in displacement thickness or circulation is the smooth, globally elliptic mode that the pressure correction removes slowest. That is the classical multigrid smoothing

argument, and it predicts the pattern this paper keeps measuring: $C_{d,v}$ recovers almost regardless of what was done to the wall gradient (§5.2.1, §5.5), $C_{d,p}$ is the laggard in every arm, refining a raster changes nothing because it does not change the smooth mode (§7.1), and a coarse-mesh solve — which attacks exactly that mode — beats every learned seed (§5.7). The §3 verification is consistent with it from a third direction: $C_{d,p}$ moves 49.8% under a fourfold change in cell count while $C_{d,v}$ moves 6.9%.

§5.9 is the one place this reading is tested rather than asserted, and it holds. A seed whose error is smooth, global and outside the boundary layer is *cheap at a loose band and expensive at a tight one*; the raster, whose error is high-wavenumber and at the wall, is the reverse. That is the prediction the smoothing argument makes, on a controlled pair, and it is what the measurement shows. We still state the reading as an interpretation rather than a result, because one crossover on five cases is evidence and not proof, because §5.9 perturbs a field rather than decomposing one, and because this paper has already withdrawn two mechanisms that were argued rather than measured — the pressure localisation of §5.2.2, and the low-pass-filtering reading of §5.1, which `scripts/mask_edge_probe.py` falsified by showing the round trip makes every seed *rougher*. Nothing in the multigrid reading survives if it is taken to mean "the projection smooths the seed": it does not. What it claims is only that the solver removes high-wavenumber error fast and smooth global error slowly, which is a property of SIMPLE and not of any seed here.

The experiment that would settle it is a seed perturbed only in its smooth, outer, elliptic content at matched norm. §5.9 runs exactly that, and its answer is consistent with the reading without establishing it: such a seed costs 12.2 points where a raster of the same error norm costs 82.4, so smooth outer error is *cheap* rather than free.

So the criterion should be used for what it measures. It says, before any solve, how badly a given output format will misreport the near-wall state, it is conservative, and it orders formats correctly. That is a genuine and cheap diagnostic of a representation. It is **not** a predictor of convergence, and §6.4's pre-flight tool must be read as reporting fidelity rather than forecasting a speedup. What *does* predict convergence here we have narrowed to the smooth, outer, elliptic content of the seed rather than its near-wall state, but we have not isolated it and we do not have a closed form for it. §10 says so.

7. What does not work, and why that matters

Three predictions a reader would reasonably make are false here, and each was tested rather than argued away. They are in the paper because a recipe that only ever confirms itself is not evidence, and because each failure is explained by the same closed form as the successes.

7.1 Refining the raster does not help, and the amount by which it does not is predicted

The obvious response to §5.2 is to spend more values. We measured it — `scripts/resolution_ladder.py`, uniform Cartesian rasters from 128^2 to 421^2 of the **exact converged field**, so surrogate accuracy is not in the comparison — and the saving is flat and negative throughout.

§6 says why, quantitatively. The predicted first-cell gradient overestimate across that ladder is

raster	values	first station	y+	predicted G
128 ²	16,384	1.17·10 ⁻²	1725	35.7×
181 ²	32,761	8.29·10 ⁻³	1220	35.7×
256 ²	65,536	5.86·10 ⁻³	863	35.7×
362 ²	131,044	4.14·10 ⁻³	610	35.7×
421 ²	177,241	3.56·10 ⁻³	525	35.5×

A 10.8-fold increase in stored values moves the predicted damage from 35.7× to 35.5× — it removes 0.5% of it. Two things make it flat, and on this ladder the second does all the work. u^+ grows logarithmically, so quadrupling the resolution buys $\ln(4)/\kappa \sim 3.4$ on a value in the thirties; and out in the wake region of the layer the velocity has saturated at freestream, so it buys nothing at all. The first four rows are in that saturated regime — their first stations sit at 63% to 19% of the boundary-layer thickness, where the log law extrapolated that far out returns a velocity *above* freestream and the u^+ cap of §6.1 takes over. So on this ladder the criterion is reporting the cruder fact that every one of these rasters puts freestream velocity in the first cell, and the law of the wall is not what is doing the work; §6.2 says where it is. The measured ladder is flat for the same reason, and the closed form turns "we tried and it did not help" into "here is the factor by which it cannot".

The scale of the gap is worth stating plainly. Resolving one cell across the inner layer on a uniform grid would need $N \approx 11,800$ — about $28\times$ beyond what the standard datasets hold and roughly 10^8 stored values. **The uniform-grid route is not expensive; it is closed.**

7.2 Seeding what the cold solver is slowest at makes it slower

Decomposing the cold solve by quantity suggests an obvious strategy. Iterations to settle within 1% of converged, cold against a seed of the exact field:

quantity	cold	oracle seed	share of C_d
viscous drag $C_{d,v}$	~700	~53	60–84%
lift C_l	~950	1	—
pressure drag $C_{d,p}$	~1850	1–2	16–40%

A cold solver is slow at pressure and fast at the near-wall velocity gradient; a surrogate is the reverse. The inference — hand over the pressure, keep the near-wall velocity — is what we pursued, and it is **false**:

- **fitted_p**, pressure only, is **inert**: +0.2% on drag, +0.1% elsewhere, at every depth.
- **composite**, potential-flow pressure plus a surrogate boundary layer, is **−305.4%**.
- **potentialFoam** alone — the free classical alternative — is inert on drag (+0.6% on $C_d@1\%$) and mildly positive on lift (+3.3%).
- The arm that wins hands over velocity and eddy viscosity inside the boundary layer and **no pressure at all**.

The reason is SIMPLE's structure [25]. Pressure is recomputed from continuity given the velocity field, so a pressure seed inconsistent with U is overwritten within a few iterations; only fields entering the momentum and turbulence transport carry information forward. We keep this section because a

falsified prediction from a measured decomposition is stronger evidence than an unfalsified one, and because it is the reason the recipe is not obvious.

7.3 The wake is worth half a per cent here

The largest acceleration reported in this literature — $26.3\times$ iterations, $16.4\times$ wall-clock [8] — comes from initialising the far wake, and every seed in this paper deliberately does the opposite, cutting off at 3.5 chords and handing the wake back to the solver. That looks like a limitation, so we bounded it rather than defending it.

`scripts/wake_probe.py` seeds the **exact converged field** across the whole downstream region — 37.5% of the cells, 21.6% of them fully — which bounds what *any* wake model could buy on these cases. Five cases, $\text{Re} = 3\cdot 10^6$:

metric	oracle wake seed	95% CI	per case
C_d,v@1%	+0.5%	[+0.4, +0.7]	+0, +0, +1, +1, +1
C_d,v@0.5%	+1.3%	[+1.0, +1.5]	+1, +1, +1, +1, +2
C_d@1%	−242.1%	[−676, −0.3]	−1098, −103, −19, −4, +13
C_l@1%	−22.0%	[−68, +1.6]	−92, +0, +2, +2

A perfect wake seed is worth half a per cent. On a 2-D attached-flow airfoil at $\text{Re} = 3\cdot 10^6$ on a 20-chord C-grid, the solver is not spending its time developing the wake; it is spending it on the near-wall state. So the two results are about different regimes, and §2.1 notes that the $26.3\times$ is itself reported as conditional on an accurate near-body field being supplied separately. Our restriction to the boundary layer is a **finding**, not a compromise, and their result and ours compose rather than compete.

It also repeats §5.4’s lesson at a different scale: the wake seed is *harmful* on total drag, because handing over a downstream field while leaving the boundary layer cold is another inconsistent pair. Consistency is not a detail of the recipe — it is most of it.

7.4 What the criterion does not do

It is necessary, not sufficient, and the study contains its own counterexample. `nf_mesh` hands over the network’s whole-field prediction at the solver’s cell centres: it satisfies the placement criterion perfectly, retains the wall gradient, and is the **worst arm in the study** at below −568% on total drag.

We do **not** attribute that to the model extrapolating outside its trust region, which is the natural reading and the one an earlier version of this paper gave. §5.3 falsifies it: grid sequencing hands over a coarse-mesh solution, which extrapolates nowhere, and its boundary-layer-restricted arm still beats its whole-field arm on every readable row. Whatever makes a whole-field seed worse than a boundary-layer-only seed is a property of seeding the outer field at all, not of the surrogate’s accuracy out there — which is condition 2, measured in §5.3 and not explained by §6. A representation that fails the criterion can be ruled out for free; one that passes it still has to satisfy conditions 2 and 3.

8. An acceptance test that bounds the worst case

Warm starting is only adoptable if a bad seed cannot cost more than not seeding. Ours can: ungated across the **repr3** tree’s 15 strategies (73 seeds, every arm, not a favourable subset), the mean is -189.0% on **C_d@1%** and the worst single seed is -1200.0% . Only 24 of the 73 seeds help at all on that metric — though on **C_d,v@1%**, the row §4 admits, 58 of 73 do, and the ungated mean is already $+20.1\%$. Which of those two pictures a gate is protecting against is the whole question, and §8’s answer is below.

The rule. Run K probe iterations from the seed. Read two scalars from the residual history — the level $\log_{10} r_K$ and the drop $\log_{10} r_K - \log_{10} r_0$. Either continue the solve, or discard the seed and start cold. Accepting costs nothing extra, because the probe iterations *are* the first K iterations of the warm solve; rejecting costs $K + \text{cold}$. The worst case is therefore $(1 + K/N_{\text{cold}}) \times \text{cold}$ **by construction**, whatever the seed does.

The rule never observes a cold run. In production there isn’t one — that is the entire point of warm starting. The threshold is calibrated leave-one-case-out and applied to the held-out case.

$K = 25$ ($\sim 3\%$ of a cold solve), threshold on the residual level, scored over every arm in the **repr3** tree:

metric	seeds	ungated mean	ungated worst	gated mean	gated worst	harmful admitted
C_d,v@1% (readable; the recommendation)	73	$+20.1\%$	-8.6%	$+21.1\%$	-8.6%	14 / 15
Cd@1%	73	-189.0%	-1200.0%	$+1.2\%$	-5.8%	0 / 49
residual 5e-6	70	-161.9%	-1449.3%	-0.5%	-7.6%	0 / 53
Cl@1%	56	$+1.1\%$	-672.6%	-8.2%	-672.6%	5 / 18

Read the two columns that matter as a pair. The gated mean is small — the gate is insurance, not a profit centre, and selling it as a mean saving would be selling the wrong product. What it does is convert a -1169.6% tail into a -5.8% one on drag, and a -1449.3% tail into -7.6% on the residual, while admitting **none** of the harmful seeds in either case.

Does the gate admit the seed this paper recommends? It is the first question to ask of an acceptance rule that sits in a paper making a recommendation, and the answer is different on the two rows, in a way that matters.

metric	nf_bl admitted	on cases where nf_bl helps	capture
C_d,v@1% (the readable row)	5 of 5	5 of 5	96.5%
C_d@1%	0 of 5	0 of 4	10.4%

On the row §4’s readability rule admits, the gate admits the recommended seed on every case — its residual level at $K = 25$ is -3.29 to -3.80 against a leave-one-case-out threshold of -2.08 to -2.26 , so it is admitted with a wide margin, and the rule’s direction is the intuitive one (a *lower* residual is accepted). On total drag the same gate **rejects nf_bl on all five cases**, four of which it would have helped by $+27.6\%$ to $+64.1\%$.

We report the second row rather than only the first because it is the sharpest version of §5.6’s finding and because a reader is entitled to it. The residual is a good proxy for viscous-drag convergence and

a poor one for total-drag convergence; a gate reading the residual therefore works on the first and misfires on the second. That is the same split as §3’s grid-convergence check ($C_{d,v}$ moves 6.9% under a fourfold cell-count change, $C_{d,p}$ 49.8%), the same split as §4’s readability rule, and the same split as §5.6’s inversion. **§8’s recommendation is therefore the $C_{d,v}$ gate**, and the C_d row is reported as a limit of the method, not as a second offering. On viscous drag, where 55 of 70 seeds already help, the gate is close to a no-op — it admits 72 of 73 seeds and catches 1 of 15 harmful ones — but the harm it lets through is bounded at -8.6% , which is what makes the no-op acceptable rather than negligent.

Where the gate fails, stated rather than buried. On lift it admits 5 of 18 harmful seeds, and its gated worst case is -672.6% — the *same* as the ungated worst, meaning it admits the single worst lift seed in the study. Lift converges by a different route — it is pressure-dominated, and §5.2 showed that resampling *helps* lift while destroying drag — so a probe reading the momentum residual is weakly informative about it. The gate should be applied per quantity, and on the quantity a user cares about; we do not claim it is a universal filter. Its worst-case bound of $(1 + K/N) \times$ cold survives regardless, because that bound is arithmetic, not statistical: it holds for any seed, any metric, and any threshold, including a threshold that admits every seed.

The gate is not what makes warm starting fast; it is what makes it deployable. On the readable metric it captures 96.5% of what a gatekeeper with foreknowledge would achieve; on total drag, where two thirds of the seeds are harmful and the residual does not track the quantity, it captures 10.4%. Longer probes are monotonically worse, and not marginally: at $K = 400$ on drag the rule admits 13 harmful seeds it rejected at $K = 25$ and returns -43.9% , because the probe cost alone (bound -49.8%) exceeds anything the decision can recover. **A short probe is not a compromise forced by cost; it is the better rule.**

9. Wall-clock

Iterations are the honest unit for a mechanism, but the claim an engineer cares about is seconds — and a mesh-native seed costs seconds a projected one does not: wall-distance computation, surrogate inference, masking, and writing the case.

`scripts/wallclock_control.py` charges each preparation stage to the arms that need it and reports **end-to-end** seconds. It runs serially and refuses to start while any other solver is up; that refusal is part of the measurement. Five cases, all arms, **exclusive: true**. `scripts/wallclock_cdv.py` re-scores those same runs on $C_{d,v}@1\%$ — the row §4’s readability rule admits and the row the headline is reported on. Every one of the five cases is readable there (settled spread 0.005–0.069% against a 0.5% limit), which the C_d version of this table cannot say of two of them.

Tree: wallclock2 (5 cases, 4 arms, **exclusive: true**).

arm		iterations	solver seconds	end-to-end seconds
oracle_mesh	(con-	+92.4%	+93.5%	+93.5% (5/5)
nf_bl	trol)	+14.6%	+16.8%	+9.0% (5/5)
fitted_bl		+41.7%	+26.4%	+26.1% (5/5)
cartesian_128		+10.0%	-1.9%	-2.3% (2/5)

The iteration saving survives translation into seconds, and the translation costs about

5.6 percentage points — almost all of it the one-off seed construction. It holds case by case rather than only on the mean:

case	iterations	end-to-end seconds	gap
naca0012@0	+19.0%	+15.7%	3.3
naca0012@4	+15.2%	+11.3%	3.9
naca0015@6	+12.6%	+4.6%	8.0
naca2412@2	+16.6%	+12.0%	4.6
naca2415@5	+9.4%	+1.4%	8.0

Seed construction is charged in full and is small against the solve, but not negligible against the saving: backbone inference at 31,700 points **10.4–10.6 s**, `wall_distance` 0.4 s, masking 0.4 s — **~11 s** against cold solves of 129–239 s, so roughly 5–8% of the solve set against a 14.6% iteration saving. On the two slowest-converging cases that leaves +4.6% and +1.4%, and we say so rather than quoting only the mean: **the recommended seed’s end-to-end advantage is real, positive on every case, and small enough that a faster or slower inference stack would move it materially.**

One detail worth stating because it cuts *for* us and could look like an error: `nf_bl` and `oracle_mesh` both do better in *solver* seconds than in iterations (+16.8% against +14.6%, +93.5% against +92.4%). A solve started near the answer has cheaper inner linear solves, so iteration counts are the conservative unit.

What an earlier version of this section reported. The same runs scored on `C_d@1%` give `nf_bl` +34.0% iterations $\rightarrow$ +28.8% seconds, which is a larger and more attractive number. It is on the row the thirteen-case corpus marks unreadable, and two of these five cases exceed the spread limit on it. We report the `C_d,v` figures above instead, and note that moving to the readable row cost us two thirds of our own headline wall-clock number.

The saving therefore survives the accounting an engineer would actually do: seconds, on one machine, with the seed’s own construction charged to it — on the metric the paper is entitled to read.

10. Limitations

What we claimed, tested, and withdrew

Five claims in this project’s history did not survive contact with a measurement. They are collected here rather than left scattered, because together they say something the individual retractions do not: **the quantity that decides a warm start here has resisted every mechanism we proposed for it, and the paper’s positive contributions are the ones that survived that.**

claim	how it died	where
<i>No 16,384-value grid has a station near the first cell</i>	arithmetic: a 64-level stack from $5 \cdot 10^{-6}$ needs growth 1.214, an ordinary mesh	\$6.3
<i>The near-wall gradient is the mediator</i>	three independent restorations — grading, wall-function repair, smoothing that repair — leave convergence unchanged or worse	\$5.2.1, \$5.5, \$6.7
<i>The damage is in the pressure field</i>	an identity ($C_d = C_{d,p} + C_{d,v}$) resting on a censored mean; the winning seed is itself -116.1% on <code>C_d,p</code>	\$5.2.2
<i>Representation, not accuracy</i>	the control that isolates accuracy prices it at -54.5 points, more than region	\$5.1
<i>A body-fitted round trip helps</i> (+16.2, 5/5 at $n = 5$)	did not survive expansion to $n = 13$ (+6.7, CI spans zero); its proposed low-pass explanation was falsified directly — the round trip makes every seed <i>rougher</i>	\$5.1

Two of these were killed by experiments we ran specifically to test our own position, and one by expanding the case set that our own protocol demanded. We report them because a paper that only

published the claims that survived would be a less reliable guide to this problem than one that shows which did not — and because §4’s rules were written from exactly these failures.

On the criterion and the closed form

- **The closed form assumes an equilibrium wall profile.** It uses the law of the wall with standard smooth-wall constants ($\kappa = 0.41$, $B = 5.0$), unmodified. Under strong adverse pressure gradients, separation, roughness, compressibility or heat transfer the profile departs from it, and the predicted factor should be read as indicative. Even on the attached cases measured here it is a bound over-predicting by 1.9-2.8x, not a point estimate.
- **It is quantitative only while the representation’s first station lies inside the boundary layer.** Above that the velocity has saturated at freestream and the expression becomes an upper bound; we report the regime alongside every number rather than leaving a reader to infer it.
- **It measures representations; it does not forecast solves (§6.7).** This is the paper’s largest limitation and it is a negative result rather than an unexplored gap. Three independent routes to a correct near-wall state — by grading (§5.2.1), by wall-function repair and by smoothing that repair (§5.5) — all leave convergence unchanged or worse. So the pre-flight check of §6.4 reports how badly a format will misreport the near-wall state, and **nothing follows from it about the speedup**. Use it to rule a format out, never to predict a gain.
- **We have narrowed where the damage is and have not isolated it.** Every projection preserves viscous drag while total drag suffers, so the damage falls outside the near-wall shear — but §5.2.2 withdraws the pressure localisation that an earlier version of this paper built on that, because it is an arithmetic identity resting on a censored mean, and because the *winning* seed is equally negative on $C_{d,p}$. What we can say is narrower: the harm lives in the seed’s smooth, outer, globally elliptic content rather than its near-wall state, and §6.7 sets out the multigrid reading that fits it. We have no closed form for that, no pre-flight test for it, and we do not claim to have identified it. §11 names the experiment that would.
- **Pressure drag is a secondary quantity here and is treated as one.** It converges three times slower than total drag from cold, its 1% row is unreadable on these trees, and the smoothed-repair observation on it (§5.5) was found after the fact. It is reported as an observation, not a result.
- **It is necessary, not sufficient (§7.4).** `nf_mesh` retains the gradient perfectly and is the worst arm in the study, so even as a veto the check has to be read alongside the region and channel conditions.
- **It is a bound, not an estimate, and it over-predicts by 1.9–2.8× (§6.2).** We do not absorb that into a fitted coefficient: fitting it would turn a prediction into a description of these five cases. An earlier version of this work reported 13% agreement; that figure was measured against a projection with a wall-distance bug and is withdrawn.

On the repair, which did not work

- **The repair restores the gradient and the solve does not follow,** and the one candidate explanation we had was tested and eliminated. The repaired seed is $29.50\times$ rougher along the wall than the converged field against $4.91\times$ for the mesh-native seed, so we smoothed the reconstruction to $6.36\times$ — matching the working seed on both diagnostics — and convergence moved 0.6 points (§5.5). We therefore do not attribute the failure to roughness, or to anything else about the near-wall velocity field.
- **The repair assumes an equilibrium profile,** as the closed form does, and assumes the rep-

representation’s first station carries a usable velocity. Where the layer separates the inverted `u_tau` is meaningless. Every case here is attached to incipiently separated, so the negative result is established only in the regime where the repair should have been at its best — which makes it a stronger negative, not a weaker one.

- **It rebuilds `nut` from a damped mixing length**, which is what Spalart–Allmaras relaxes to in the log layer but not what SA solves, and §5.4 shows `nut` is the least forgiving channel. We cannot exclude that this, rather than the gradient reconstruction, is what the solver objected to.

On the study

- **2-D, incompressible, steady, one turbulence model.** Spalart–Allmaras only. Whether the three conditions survive a two-equation model is untested, and $k\text{-}\omega$ SST is the obvious next experiment.
- **One solver and one mesh family.** OpenFOAM SIMPLEC on a C-grid we generate. Nothing here has been tried on an unstructured or commercial solver.
- **One Reynolds number for the *solver* results** ($\text{Re} = 3 \cdot 10^6$). The convergence savings, the conditions and the repair are all measured there and nowhere else. The *mechanism* is checked across $\text{Re} = 10^3$ to $3 \cdot 10^6$ in §6.6, but that check is on the seed’s wall gradient, not on convergence: it does not show that a warm start helps at another Reynolds number, only that the damage a representation does behaves as the closed form says it will.
- **The closed form is accurate only where the boundary layer is turbulent and attached.** §6.6 measures agreement of 0.83–0.96 at $\text{Re} \geq 10^6$ and 0.32–0.35 at $\text{Re} \leq 10^5$, where 42–62% of the surface carries near-zero wall shear. There it is a lower bound rather than an estimate — conservative, but not quantitative.
- **Bands below 1% need a longer budget.** At 0.5% and 0.2% the total-drag rows are unreadable at 6000 iterations on the corpus.
- **`nuTilda` is floored at freestream on write**, a common-mode limitation quantified in §5.8 — it removes 2.1–2.3% of the eddy-viscosity field’s energy in the boundary layer and applies identically to every arm including the oracle.
- **Three cases have no unique steady drag at this budget**, and they share a shape: `naca4412@3deg`, `naca4415@2deg`, `naca4415@4deg` — thick cambered sections at low incidence, carrying the corpus’s worst residual floors. We do not know whether this is genuine non-uniqueness or a budget we did not pay, and we do not claim to.
- **The residual metric is negative for the recommended arm** (§5.6). We report it every time we report the force metric.
- **Wall-clock is $n = 5$ and was measured exclusively; the placement, repair and sequencing trees were not.** Those three were run concurrently, so iterations from them are sound — iteration counts are contention-proof — and **no wall-clock number is quoted from them.**
- **The acceptance test fails on lift** (§8): its gated worst case equals its ungated worst case, so it admits the single worst lift seed in the study. The $(1 + K/N)$ bound is arithmetic and survives that, and §7.2 explains why lift behaves differently from drag.

11. Conclusion

Whether a neural surrogate can accelerate a production RANS solve is decided by three properties of what it hands over, and they can be set independently. The cleanest statement needs no network at all: take the exact converged flow field and give it back to the solver in variants that differ one property at a time. On viscous drag at a 1% band — the row this study’s readability rule admits:

what moves	effect on $C_{d,v@1\%}$	
read at the solver's own cell centres	+93.6% [+92.9, +94.3]	13/13
→ stored on a 128^2 raster of 16,384 values	−90.2 points [−96.3, −84.7]	0/13
→ stored on a body-fitted grid of the same 16,384 values	+6.7 [−2.6, +14.8]	<i>no measurable cost</i>
→ restricted to the boundary layer	−20.7 points [−24.2, −17.6]	0/13
→ the surrogate's prediction instead of the exact field	−54.5 points [−58.9, −49.7]	0/13

Every row is the same thirteen cases and the same tree, paired within case.

A perfect flow field on a raster is worth no initialisation, and a perfect flow field on a body-fitted grid of identical budget is worth nearly everything. So the property that matters is not "a grid" and not the number of values: it is whether the format places a sample where the solver keeps its state.

It is not the *size* of the error either, and that is worth stating separately because it is the objection this design invites. An oracle seed is a fixed point of the discrete operator, so a contrast against it could be measuring distance from that fixed point rather than representation. §5.9 perturbs the converged field by a smooth error carrying the raster's **own** whole-field norm, ramped to zero inside the boundary layer. At the 1% band it costs **12.2 points**; the raster, with the identical error norm, costs **82.5**. Six sevenths of the harm is in *where* the error sits.

That control also produced the one thing this paper had argued for and never measured. The smooth seed is far better than the raster at a loose band and far *worse* at tight ones — cheap early, expensive late — which is what a SIMPLE-family solver does to a smooth elliptic mode, and the mirror image of what it does to the raster's high-wavenumber near-wall error. §6.7's reading made that prediction; §5.9 is where it is tested.

What a representation does to the near-wall state we can state in closed form. A resampled field hands every cell nearer the wall than its first station that station's value, so the first-cell wall gradient is overestimated by $u+(y1+)/u+(y_c+)$ — no fitted parameter, an upper bound on the measurement wherever the mechanism is active, and from $Re = 10^3$ to $3 \cdot 10^6$. It indicts placement rather than budget, and directly: cutting the stored values fourfold moves the damage by a factor of 1.00 while moving the first station moves it $12.7\times$.

And that is not what costs the solve. The arm carrying 1218.8% gradient error is the *best* of its ladder on the readable row. Repairing the gradient by wall function moves convergence by a tenth of a point; smoothing that repair to the working seed's own roughness moves it half a point more. Three independent routes to a correct near-wall state, and none of them recovers the solve. We had predicted the opposite and registered the prediction before running it.

Read as advice rather than as a result, the paper is short:

Query your surrogate where the solver lives — and if you cannot, put your representation's first station inside the mesh's first cell, which is a grading choice and not a budget. Give the solver only the region your surrogate is trusted on, hand over whole physics rather than single channels, and spend 3% of a solve checking before you commit. Then check whether grid sequencing, which needs no network, already does better.

The size of the effect is modest and we say so. The recommended seed accelerates viscous-drag

convergence by 18.4% across thirteen cases, winning every one ($p = 0.0002$), with a converged-field control at 93.6% and a null negative control. That is far short of the $26.3\times$ reported elsewhere for a different regime, and short of what classical grid sequencing achieves here on the same cases. What is durable is not the number.

What is durable is that the representation result needs no network to state, and that the paper’s own explanations of it were tested rather than asserted — three times to destruction. We predicted the near-wall gradient was the mediator, built three ways to restore it, and reported that all three failed. We then proposed the pressure field, and withdrew that too when the arithmetic behind it turned out to be an identity resting on a censored mean. And we claimed representation mattered rather than accuracy until the control that isolates accuracy showed it costs 55 points. The criterion that survives all of this is a cheap, conservative, correctly-ordered diagnostic of what a representation *keeps* — and not a forecast of a speedup. Naming the difference between those two is, we think, the more useful contribution.

CRedit authorship contribution statement

Ali Jabbari: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, Writing – review & editing, Supervision. **Kasra Ghanavati:** Validation, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

All code, meshes, solver configurations and result files for this paper are openly available in the tagged snapshot <https://github.com/ali-kin4/neuroforge-cfd/tree/paper2-v1>, archived at [DOI: to be inserted at submission]. **The tag matters: this work lives on a branch, and the repository’s default branch does not contain it.** Every table and figure is regenerated by a single command from checkpointed solver output; Appendix A gives the mapping from each table to the script and result file that produces it. The flow solver is OpenFOAM v2606 (ESI), used unmodified. The surrogate is trained on the public AirFRANS dataset.

Appendix A – reproduction

Each result maps to one script and one committed result file.

result	script	result file
Mesh and solver verification (§3, Table 1)	<code>verify_mesh.py</code>	<code>mesh_verification.json</code>
Wall-gradient diagnostic (§5.2.1, §5.5)	<code>seed_gradient_diagnostic.py</code>	<code>seed_gradient_placement.json</code> , <code>seed_gradient_repair.json</code>
Three conditions (§5.2–§5.4)	<code>mesh_native_probe.py</code>	<code>depth_repr3_nowake.json</code>
Placement ladder (§5.2.1)	<code>placement_probe.py</code>	<code>depth_placement.json</code> , <code>depth_placement2.json</code>
Repair tree (§5.5)	<code>repair_probe.py</code>	<code>depth_repair.json</code>
Grid sequencing (§5.7)	<code>sequencing_probe.py</code>	<code>depth_sequencing.json</code>
Thirteen-case corpus (§5.1, §5.2)	<code>corpus_probe.py</code>	<code>depth_corpus.json</code>
Closed-form validation (§6.2, Table 2)	<code>validate_closed_form.py</code>	<code>closed_form_validation.json</code>
Criterion tables (§6.5, §7.1; Tables 3–4)	<code>criterion_tables.py</code>	<code>criterion_tables.json</code>
Reynolds transfer (§6.6)	<code>reynolds_transfer.py</code>	<code>reynolds_transfer.json</code>
Pre-flight check (§6.4)	<code>preflight.py</code>	– (a CLI; no stored result)
Acceptance certificate (§8)	<code>certificate.py</code>	<code>cert_all13_*.json</code>
Wall-clock (§9)	<code>wallclock_control.py</code>	<code>wallclock_control.json</code>
Wake bound (§7.3)	<code>wake_probe.py</code>	<code>wake_probe_*.json</code>
Figure 1	<code>plot_mechanism.py</code>	<code>mechanism.png</code>
Figure 2	<code>plot_bands.py</code>	<code>bands.png</code>
Figure 3	<code>plot_placement.py</code>	<code>placement.png</code>

Re-scoring any finished tree at every convergence depth and force band is a single command, `reanalyse_depth.py`, which also declares the arm set it scored over and prints the readability verdict for every row.

Appendix B – the scoring rules as code

The eight rules of §4 are implemented in `solver/scoring.py` and `scripts/reanalyse_depth.py` as `has_settled`, `settled_reference`, `bounded_saving`, `shared_reference`, `reference_spread`, `readable_depth`, `scoreable_budgets`, `bootstrap_ci` and `sign_test`, with `MIN_DEPTH_OVER_FLOOR = 5.0` and `MAX_SPREAD_FRACTION = 0.5`. Each carries a unit test named for the mistake it prevents.

References

- [1] F. Bonnet, A. J. Mazari, P. Cinnella, P. Gallinari. AirFRANS: high-fidelity computational fluid dynamics dataset for approximating Reynolds-averaged Navier–Stokes solutions. *Advances in Neural Information Processing Systems* 35 (2022). arXiv:2212.07564.
- [2] H. Wu, H. Luo, H. Wang, J. Wang, M. Long. Transolver: a fast transformer solver for PDEs on general geometries. *International Conference on Machine Learning* (2024). arXiv:2402.02366.
- [3] C. Zeng, Y. Zhang, J. Zhou, et al. Point cloud neural operator for parametric PDEs on complex and variable geometries (2025). arXiv:2501.14475.
- [4] M. S. Eshaghi, C. Anitescu, N. Valizadeh, Y. Wang, X. Zhuang, T. Rabczuk. Nows: neural operator warm starts for accelerating iterative solvers. *Computer Methods in Applied Mechanics and Engineering* 458 (2026) 118989. arXiv:2511.02481.
- [5] J. Oh, Y. Lee, J. Darbon, et al. Spectrally safe neural operator warm-starts for large-scale Newton solvers (2026). arXiv:2606.21828.
- [6] M. Khodak, M. K. Jung, B. Wynne, et al. One-shot acceleration of transient PDE solvers via online-learned preconditioners (2025). arXiv:2509.08765.
- [7] P. Sharpe, R. Ranade, K. Tangsali, et al. Accelerating transient CFD through machine learning-based flow initialization (2025). arXiv:2503.15766.

- [8] K. W. Fuchi, E. M. Wolf, C. R. Schrock, P. S. Beran. Acceleration of RANS solver convergence via initialization with wake extension models (2025). arXiv:2501.14699.
- [9] I. Yavlovich, J. Agbaria, M. Mhamed, et al. Learning-augmented scalable linear assignment problem optimization via neural dual warm-starts (2026). arXiv:2605.09382.
- [10] P. R. Spalart, S. R. Allmaras. A one-equation turbulence model for aerodynamic flows. *La Recherche Aeronautique* 1 (1994) 5–21.
- [11] H. G. Weller, G. Tabor, H. Jasak, C. Fureby. A tensorial approach to computational continuum mechanics using object-oriented techniques. *Computers in Physics* 12 (6) (1998) 620–631.
- [12] X.-H. Zhou, J. Han, M. I. Zafar, E. M. Wolf, C. R. Schrock, C. J. Roy, H. Xiao. Neural operator-based super-fidelity: a warm-start approach for accelerating steady-state simulations. *Journal of Computational Physics* (2025). arXiv:2312.11842.
- [13] T. Hu, C. Wu, J. Ding, X. Wang, Y. Yang, J. Wang. Data-driven flow initialization framework for CFD acceleration of underwater vehicle in vertical-plane oblique motion (2026). arXiv:2601.02693.
- [14] T. A. Mehta, P. S. Bhati, H. D. Akolekar. DD-RNO: a domain-decomposed routed neural operator for airfoil flow prediction (2026). arXiv:2608.13490.
- [15] X. Zhang, Y. Huang, S. Jiang, et al. Geometry-aware anisotropic boundary correction for aerodynamic simulation (2026). arXiv:2606.09963.
- [16] Z. Yang, H. Xin, T. Du, et al. Simple yet effective: low-rank spatial attention for neural operators (2026). arXiv:2604.03582.
- [17] Z. Zhang, X. Yang, Y. Miao, et al. PGOT: a physics-geometry operator transformer for complex PDEs (2025). arXiv:2512.23192.
- [18] E. J. Schmidtobreck, D. Arnstroem, P. Haeusner, et al. Warm-starting active-set solvers using graph neural networks (2025). arXiv:2511.13174.
- [19] P. Araujo da Cunha Sousa, A. M. Afonso, C. Veiga Rodrigues. Surrogate-based pressure–velocity coupling: accelerating incompressible CFD flow solvers with machine learning. *Computers & Fluids* (2026), available online July 2026.
- [20] J. Scherz, D. Hines, P. Bekemeyer. Evaluation of state-of-the-art deep learning architectures for aerodynamical predictions (2026). arXiv:2607.13866.
- [21] M. Yagoubi, D. Danan, M. Leyli-Abadi, et al. NeurIPS 2024 ML4CFD competition: results and retrospective analysis (2025). arXiv:2506.08516.
- [22] K. W. Fuchi, E. M. Wolf, D. S. Makhija, et al. Multi-fidelity machine learning applied to steady fluid flows (2025). arXiv:2501.14870.
- [23] Y. Liu, H. Wang, Y. Qi, et al. Full-field prediction for engineering-scale three-dimensional aircraft with multigrid-hierarchical learning (2026). arXiv:2605.30375.
- [24] Y. Yang, B. Gholami, C. Gurbuz, et al. Towards a foundation-model paradigm for aerodynamic prediction in three-dimensional design (2026). arXiv:2604.18062.

- [25] S. V. Patankar, D. B. Spalding. A calculation procedure for heat, mass and momentum transfer in three-dimensional parabolic flows. *International Journal of Heat and Mass Transfer* 15 (10) (1972) 1787–1806.
- [26] Y. Liu, W. Zhang, J. Kou. Mode multigrid – a novel convergence acceleration method (2018). [arXiv:1802.08962](https://arxiv.org/abs/1802.08962).